Fase 5 — División del trabajo entre 2 desarrolladores (trabajo secuencial)

Guía de trabajo para construir el **Punto 5 (pesaje final y conciliación)** y cerrar el **circuito completo 1 → 4** entre dos personas, **una después de la otra**, en la misma rama. Dice dónde está parado cada uno respecto de la Fase 4, qué tiene que hacer, cómo hacerlo, cómo probarlo y trae el prompt para usar con Claude Code.

- **Fecha:** 02/11/2026 · **Rama:** `fase-5-conciliacion` (sale de `main` actualizado, que ya incluye las Fases 0 a 4: la Fase 4 entró por el PR #12 el 31/10/2026)
- **Planeamiento autorizado por Martín González** el 02/11/2026. Las **decisiones D1 a D14** (sección 3) quedaron **confirmadas** ese día, y también los **commits locales atómicos** y los dos arreglos de las Fases 2 y 3 (D13 y D14).
- **Primera entrega del TP:** 10/11/2026.
- **Orden de trabajo:** primero Calvo, después Alasino. Cada uno empieza cuando el anterior **publicó** su parte.

Parte	Responsable	Usuario de GitHub	Qué construye
1	Tomás Calvo	`tomas-calvo01`	**Negocio y endpoints**: pesaje final, conciliación y los dos arreglos (H-25 y H-26)
2	Benjamín Alasino	`benjaalasino`	**Pruebas, Postman 05 y 06, concurrencia, auditoría final**, documentación y PR
—	Martín González	`mgonzalez309-dev`	No programa en esta fase: publica este documento y **revisa y aprueba el PR**
—	Joaquín Contreras	`joacoContreras`	No programa en esta fase: **revisa y aprueba el PR**

Cómo leer este documento. Todos leen las secciones 1 a 9 (contexto, contrato y forma de trabajo). Después, cada uno lee **su parte** y copia **su prompt**.

---

1. Dónde estamos parados (lo que dejó la Fase 4)

La Fase 4 construyó el Punto 4: el sistema de carga cierra la orden y esta pasa al **estado 3**; desde ahí las lecturas se rechazan con 409. Todo esto ya existe y se reutiliza (verificado en el código de `main`):

Ya existe	Dónde	Cómo se usa en la Fase 5
La orden con `tare`, `initialWeighingDateTime`, `activationDateTime`, `loadingStartDateTime`, `loadingEndDateTime` y los cuatro `last*`	`model/Order.java`	`tare` es el pesaje inicial y `lastAccumulatedMass` es el "producto cargado" de la conciliación
El enum de estados con `CLOSED_FOR_LOADING` (3) y `FINISHED` (4)	`model/OrderStatus.java`	`FINISHED` todavía no lo usa ninguna operación: la Fase 5 lo estrena
El histórico de lecturas	`model/OrderDetail.java` (tabla `order_details`) y `model/persistence/OrderDetailRepository.java` (hoy solo `findByOrder`)	Se le suma **una** consulta con `AVG`
Buscar una orden con bloqueo de fila	`OrderRepository.findByOrderNumberForUpdate` (`PESSIMISTIC_WRITE`) y el `loadForUpdate` privado de `OrderBusiness`	El pesaje final los reutiliza; **no** se crea un tercer duplicado
El molde de una transición de estado	`OrderBusiness.close` (`loadForUpdate` → comprobaciones fuera del `try` → `saveAndFlush` → `logStatus`)	Es el molde de `registerFinalWeight`
`logStatus`	`OrderBusiness` (privado)	Escribe la fila 3→4

Ya existe	Dónde	Cómo se usa en la Fase 5
El controller de la balanza, con el tag "Pesaje inicial y final de la orden"	`integration/tms/controllers/OrderTmsRestController.java` (hoy solo `/tare`) y `integration/tms/model/` (`TareRequest`, `TareResponse`)	Se agrega ahí el pesaje final
El controller de consultas	`controllers/OrderRestController.java` (ya inyecta `IOrderBusiness`)	Se agrega ahí la conciliación
Excepciones *checked*	`business/exception/` (`NotFound`, `Found`, `BadRequest`, `Business`, `Conflict`, `Forbidden`)	No hace falta ninguna nueva
Un test con Mockito	`src/test/java/ar/edu/iua/iw3/business/OrderBusinessCloseTest.java` (7 tests, `@InjectMocks OrderBusiness`)	Molde de los tests nuevos
Postman	`docs/postman/IW3-Gas-Terminal.postman_collection.json`: **86 pedidos y 204 aserciones** (carpetas 00 a 04)	Se agregan las carpetas 05 y 06 con **variables propias**
Tests de Maven	**19** (`ActivationPasswordGeneratorTest` 2, `OrderDetailValidatorTest` 9, `OrderBusinessCloseTest` 7, `Iw3ApplicationTests.contextLoads` 1)	Se suman los nuevos

Lo que NO existe todavía y se construye ahora: los campos del pesaje final, la consulta de promedios, la clase de la conciliación, el pesaje final con su transición 3 → 4, la consulta de la conciliación, las carpetas 05 y 06 de Postman, la auditoría final del parcial, y los arreglos de la tara infinita (H-26) y de la activación sin bloqueo (H-25).

2. Qué hay que construir (la Fase 5 en una página)

La **balanza (TMS)** pesa el camión cargado y manda el **pesaje final**. El backend:

1. **Recibe** el peso, **pasa la orden de 3 a 4** y deja el cambio en el historial.
2. **Devuelve la conciliación** con los valores calculados (el POST ya la responde).
3. Permite **consultarla** más tarde con un endpoint específico, **solo para órdenes en estado 4**.

```
Balanza ──POST /api/v1/integration/tms/orders/{n}/final-weight { weight }──►
   OrderTmsRestController ──► IOrderBusiness.registerFinalWeight (Parte 1) ──► Order (estado 4) +
order_status_logs (3→4)
◄── 200 { conciliación }

Cualquier cliente ──GET /api/v1/orders/{n}/reconciliation──►
   OrderRestController ──► IOrderBusiness.reconciliation (Parte 1) ──► calcula con Order + AVG de order_details
◄── 200 { conciliación }   (solo en estado 4; en otro estado, 409)
```

La conciliación **no se guarda**: se calcula cada vez, con **una sola función** para el POST y el GET.

Además, la **Parte 2** cierra el segundo parcial: prueba el **circuito completo 1 → 4** con *Tests* de Postman encadenados y **audita** que estén cubiertos todos los requisitos obligatorios contra el PDF oficial.

2.1 Requisito de la consigna o decisión nuestra

La regla del equipo es distinguir una cosa de la otra (`CLAUDE.md`, sección 3, regla 5). **La consigna oficial está en el repo** (`docs/pdf/consigna-trabajo-integrador-2026.pdf`, 9 páginas) y se contrastó contra este documento: el Punto 5 (página 3), el anexo de la orden (páginas 4 y 5), los estados y las notas (páginas 6 y 7) y los requisitos del parcial y del final (páginas 8 y 9). No se encontraron contradicciones con `CLAUDE.md` sección 4.

Qué	Fuente
El sistema de balanza envía el pesaje final de la orden cerrada	Consigna, Punto 5 (p. 3)
El backend recibe el pesaje, cambia la orden a estado 4 y devuelve la conciliación	Consigna, Punto 5

Qué	Fuente
Datos de la conciliación: pesaje inicial (tara), pesaje final, producto cargado (último valor de masa acumulada), neto por balanza (final − inicial), diferencia (neto por balanza − producto cargado) y promedios de temperatura, densidad y caudal "según datos almacenados"	Consigna, Punto 5
La conciliación se pide en cualquier momento, **solo para órdenes en estado 4**, con un endpoint específico	Consigna, Punto 5
Fecha y hora de pesaje final = "momento de recepción del pesaje final"	Consigna, anexo de la orden (p. 4)
Los cambios de estado son secuenciales y unidireccionales y cada uno queda registrado (fecha y hora, usuario o proceso, observaciones)	Consigna, notas (p. 7)
Colección Postman que pruebe todas las APIs; documentación OpenAPI/Swagger; APIs sin interfaz gráfica	Consigna, requisitos del segundo parcial (p. 8)
Simulación del circuito completo con *Tests* de Postman	Consigna, opcional del parcial (p. 8; el PDF dice "hasta el cierre": nuestra carpeta 06 llega al estado 4, que lo incluye)
Promedios (opcional en el PDF: "podrá almacenarse simplemente el último valor")	Consigna, opcional; **decisión 3 del plan** la convierte en obligatoria para nosotros
409 para estado incorrecto, bloqueo de fila, qué se guarda, qué se calcula y cómo se redondea, orden sin lecturas, peso menor que la tara, etc.	**Decisiones nuevas D1 a D14** (sección 3)

3. Decisiones confirmadas y contrato compartido

Confirmadas por Martín el 02/11/2026. No se cambian sin avisar a los dos. Al cerrar la fase, Alasino las registra en el plan como decisiones 33 a 46 (D1 → 33, D2 → 34, ..., D14 → 46).

#	Decisión	Tema	Resolución
D1	33	Estado de partida	`final-weight` solo desde el **estado 3**; los estados 1, 2 y 4 dan **409**. **No es idempotente**: el segundo pesaje da 409
D2	34	Validaciones del peso y orden	**404** → **400** → **409** → **400**: (1) orden inexistente; (2) peso ausente, no finito o ≤ 0; (3) estado distinto de 3; (4) peso **menor que la tara**. Un peso **igual** a la tara es válido (neto 0). La comprobación contra la tara va **después** del estado: en el estado 1 la tara es nula
D3	35	Concurrencia y transacción	La orden se busca con `findByOrderNumberForUpdate` (bloqueo pesimista), como el cierre; comprobaciones **fuera del** `try`; `@Transactional(rollbackFor = Exception.class)`; `saveAndFlush`. El `GET` no bloquea
D4	36	Qué se guarda	Solo `finalWeight` y `finalWeighingDateTime` (**hora del servidor**; el cliente no manda fecha). La conciliación **no se guarda**: se calcula al consultarla, con una sola función para el POST y el GET
D5	37	Cálculos	Producto cargado = `lastAccumulatedMass`; neto por balanza = `finalWeight − tare`; diferencia = neto por balanza − producto cargado; promedios = `AVG` **simple** (no ponderado por tiempo) sobre `order_details` (decisión 3). **Sin redondeo** en el backend: son `double` y pueden traer residuos binarios (por ejemplo `18400.2 − 14500.1` da `3900.1000000000004`); el cliente formatea y Postman compara con `closeTo`
D6	38	Orden sin lecturas	Posible por la decisión 30. Producto cargado = **0.0** y promedios `null`; **no** bloquea el pesaje final (si se rechazara, la orden quedaría trabada en el estado 3 sin salida)
D7	39	Respuestas	El POST responde **200 con la conciliación** (el mismo objeto que el GET). El GET solo en estado 4: otros estados **409**, inexistente **404**, sin token **401**
D8	40	Historial	Una fila **3** → **4** con el usuario del token y la observación `Pesaje final registrado`. El estado 4 es **final**: no se reabre
D9	41	Ubicación	La lógica en `IOrderBusiness` / `OrderBusiness` (con `OrderDetailRepository` inyectado **directo**: inyectar `IOrderDetailBusiness` sería una dependencia circular). El POST en `OrderTmsRestController`; el GET en `OrderRestController`. **No hace falta tocar** `Constants` (ambas rutas cuelgan de las URL existentes)

#	Decisión	Tema	Resolución
D10	42	Mensajes	Estilo de las fases anteriores (sección 3.6). Las aserciones de Postman usan **fragmentos** (`include`)
D11	43	Postman	Carpeta **05** y carpeta **06**, con variables propias; no se tocan las carpetas 00 a 04. Línea base a no romper: **86 pedidos, 204 aserciones y 19 tests** de Maven
D12	44	Fuera de alcance	Frecuencia de almacenamiento configurable (opcional del parcial), roles, alarma por e-mail, frontend y seguridad (examen final)
D13	45	Arreglo H-26 (Fase 2)	La **tara** también rechaza un peso infinito con **400** (hoy `1e999` da 500 con el formato de Spring). Un helper privado `requireValidWeight` sirve a la tara y al pesaje final. El mensaje de "≤ 0" **no cambia** (la carpeta 02 de Postman lo comprueba)
D14	46	Arreglo H-25 (Fase 3)	`activate` busca la orden **con bloqueo** (`loadForUpdate`) después de comprobar el cuerpo. Así una activación simultánea con el cierre no puede dejar la orden en el estado 2. Efecto colateral: también resuelve **H-16** (activaciones simultáneas)

> **Sobre el "decisión 11".** En `CLAUDE.md` (sección 9) la decisión 11 es "permitir `/error` y corregir el `produces` del login"; en el documento de la Fase 3 "decisión 11" era el histórico paginado (la decisión **28** global). Este documento usa **siempre el número global** (33 a 46) para evitar la confusión.

3.1 Modelo de datos

`Order` (cabecera): campos nuevos, opcionales (la tabla `orders` ya tiene filas y `ddl-auto=update` solo agrega columnas), con `@Schema(accessMode = READ_ONLY)` :

Campo	Tipo	Columna	Significado
`finalWeight`	`Double`	`final_weight`	Peso del camión cargado (kg), enviado por la balanza. Nulo hasta el pesaje final
`finalWeighingDateTime`	`LocalDateTime`	`final_weighing_date_time`	Momento de recepción del pesaje final (hora del servidor). Nulo hasta entonces

`OrderDetailRepository` : una consulta nueva

```
@Query("select avg(d.temperature) as temperature, avg(d.density) as density, avg(d.flowRate) as flowRate "
     + "from OrderDetail d where d.order = ?1")
ReadingAverages averagesByOrder(Order order);
```

`ReadingAverages` es una **interfaz de proyección** (en `model/persistence`) con `Double getTemperature()` , `Double getDensity()` y `Double getFlowRate()` . Los alias de la consulta tienen que coincidir con los getters. **A verificar:** con cero lecturas, el `AVG` devuelve una fila con nulos; si el repositorio devolviera `null` , se trata igual (promedios `null`).

3.2 Endpoints

Método y ruta	Cuerpo	Respuestas
`POST /api/v1/integration/tms/orders/{orderNumber}/final-weight`	`{ "weight": 16490.0 }`	**200** conciliación · 400 · 401 · 404 · **409** · 500
`GET /api/v1/orders/{orderNumber}/reconciliation`	—	**200** conciliación · 401 · 404 · **409** · 500

3.3 Clases y firmas (nombres exactos)

Pieza	Paquete (bajo `ar.edu.iua.iw3`)	Detalle
`Reconciliation`	`business`	Clase pública con `@Getter`, `@AllArgsConstructor` y `@Schema`: `Integer orderNumber`, `Double tare`, `Double finalWeight`, `Double loadedProduct`, `Double scaleNet`, `Double difference`, `Double averageTemperature`, `Double averageDensity`, `Double averageFlowRate`. Es el cuerpo de la respuesta de los dos endpoints (como `DetailResult`, vive en `business`)
`ReadingAverages`	`model.persistence`	Interfaz de proyección (sección 3.1)
`OrderDetailRepository.averagesByOrder`	`model.persistence`	Sección 3.1
`IOrderBusiness.registerFinalWeight`	`business`	`Reconciliation registerFinalWeight(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException`
`IOrderBusiness.reconciliation`	`business`	`Reconciliation reconciliation(Integer orderNumber) throws NotFoundException, ConflictException, BusinessException`
`FinalWeightRequest`	`integration.tms.model`	`double weight` **primitivo**, como `TareRequest`: un campo ausente llega como `0.0` y se rechaza con 400
`OrderTmsRestController.registerFinalWeight`	`integration.tms.controllers`	`@PostMapping("/{orderNumber}/final-weight")`, mismo estilo que `registerTare`
`OrderRestController.reconciliation`	`controllers`	`@GetMapping("/{orderNumber}/reconciliation")`

3.4 Reglas, en orden

Pesaje final (`registerFinalWeight`), `@Transactional(rollbackFor = Exception.class)`, comprobaciones **fuera del `try`**:

1. `loadForUpdate(orderNumber)` → si no existe, **404**.
2. `requireValidWeight(weight)`: `!(weight > 0)` (también rechaza `NaN`) → **400**; infinito → **400** (texto distinto).
3. Estado distinto de `CLOSED_FOR_LOADING` → **409**.
4. `tare` nula (no debería pasar en el estado 3) → **409**.
5. `weight < tare` → **400**. (`weight == tare` es válido.)
6. Dentro del `try`: `from = status`; `finalWeight = weight`; `finalWeighingDateTime = now()`; estado `FINISHED`; `saveAndFlush`; `logStatus(order, from, to, actor, "Pesaje final registrado")`; devolver la conciliación armada con la función compartida.

Consulta (`reconciliation`), sin bloqueo:

1. `load(orderNumber)` → **404**.
2. Estado distinto de `FINISHED` → **409**.
3. Armar la conciliación con la función compartida (un error técnico sale como `BusinessException`).

Función compartida (`buildReconciliation`, privada):

Campo	Cálculo
`tare`	`order.tare`
`finalWeight`	`order.finalWeight`
`loadedProduct`	`lastAccumulatedMass`, o `0.0` si es nulo (D6)
`scaleNet`	`finalWeight − tare`
`difference`	`scaleNet − loadedProduct`
`averageTemperature`, `averageDensity`, `averageFlowRate`	`averagesByOrder(order)`; `null` si no hay lecturas

Tara (`registerTare`, D13) y activación (`activate`, D14): solo cambian en lo indicado en las decisiones; el resto de sus reglas **no se toca**. En `activate` el orden sigue siendo cuerpo (400) → orden (404) → estado (409) → contraseña (403), pero la orden se busca con bloqueo.

3.5 Ejemplo numérico (para tests y Postman)

Valores **exactamente representables** en `double`, para que las comparaciones sean exactas.

Dato	Valor
Tara (pesaje inicial)	`14500.0`
Lectura 1	masa `1000.0`, densidad `0.5`, temperatura `20.0`, caudal `3000.0`
Lectura 2	masa `2000.0`, densidad `0.5`, temperatura `22.0`, caudal `3600.0`
Pesaje final enviado	`16490.0`

Resultado: `loadedProduct` **2000.0**, `scaleNet` **1990.0**, `difference` **−10.0**, `averageTemperature` **21.0**, `averageDensity` **0.5**, `averageFlowRate` **3300.0**.

```
{ "orderNumber": 90001, "tare": 14500.0, "finalWeight": 16490.0, "loadedProduct": 2000.0,
  "scaleNet": 1990.0, "difference": -10.0, "averageTemperature": 21.0,
  "averageDensity": 0.5, "averageFlowRate": 3300.0 }
```

3.6 Mensajes del contrato

Alasino arma las aserciones de Postman sobre **fragmentos** de estos textos (con `include`), no sobre el texto completo.

Caso	Código	Mensaje
Orden inexistente	404	`No se encuentra la Orden numero=<n>`
Peso ausente, 0, negativo o `NaN`	400	`weight debe ser mayor que 0 (kg)` (el mismo texto de la tara)
Peso infinito (tara o pesaje final)	400	`weight debe ser un número finito`
Peso menor que la tara	400	`weight (pesaje final) no puede ser menor que la tara (<t> kg)`
Pesaje final fuera del estado 3	409	`La orden <n> no admite pesaje final: debe estar en estado 3 (estado <e>)`
Orden en estado 3 sin tara	409	`La orden <n> no tiene tara registrada`
Conciliación fuera del estado 4	409	`La conciliación solo está disponible para órdenes en estado 4 (orden <n> en estado <e>)`
Observación del historial 3 → 4	—	`Pesaje final registrado`

4. Cuidados y trampas (leelos antes de empezar)

1. **El orden de D2 no es el de las fases anteriores.** Hay un 400 *después* del 409: la comparación con la tara necesita la tara, y en el estado 1 la tara es nula (se rompería con un `NullPointerException` y saldría 500).
2. **Peso igual a la tara es válido.** Con la decisión 30 una orden se cierra sin lecturas; si el camión no cargó nada, su peso final es igual a su tara. Rechazar `<=` dejaría esa orden trabada en el estado 3.
3. **El peso infinito (H-26).** `1e999` pasa `!(w > 0)` y MySQL lo rechaza al guardar: 500 con el formato de Spring y con el `trace`. Probado el 02/11/2026 contra la tara; la orden queda en estado 1. `requireValidWeight` lo evita **para la tara y para el pesaje final**. El mensaje de "≤ 0" **no se cambia**: la carpeta 02 de Postman lo comprueba.
4. **Comprobaciones fuera del `try`.** Si el `catch (Exception)` las envolviera, un 404, 400 o 409 saldría como 500.
5. **`@Transactional(rollbackFor = Exception.class)` y `saveAndFlush`.** Las excepciones de negocio son *checked* (Spring no revierte por ellas) y `save` deja el `UPDATE` para el `COMMIT`, fuera del `try` (lección de H-14).
6. **Reutilizá `loadForUpdate`.** `OrderBusiness` ya lo tiene. No agregues un tercer duplicado ni refactorices `OrderDetailBusiness`.
7. **No inyectes `IOrderDetailBusiness` en `OrderBusiness`.** `OrderDetailBusiness` ya depende de `IOrderBusiness`: sería una dependencia circular y el contexto no arranca. Inyectá `OrderDetailRepository`.
8. **La consulta de promedios.** Los alias tienen que coincidir con los getters de la proyección. Con cero lecturas hay que obtener promedios nulos, no una excepción.
9. **Residuos de `double`.** No redondees. Los ejemplos de la sección 3.5 usan valores exactos; para otros valores, Postman compara con `closeTo`.
10. **El promedio es simple.** Es el `AVG` de las filas guardadas, no un promedio ponderado por tiempo. Dejalo escrito en el informe.
11. **Una sola función para el POST y el GET.** Si calculás dos veces, los resultados pueden divergir.
12. **La conciliación no se persiste.** No agregues columnas para los valores calculados.
13. **El estado 4 es final.** Tras el pesaje final, lecturas, activación, cierre, tara y un segundo pesaje deben dar **409 sin tocar su código**. Se verifica, no se programa.
14. **Los campos nuevos de `Order` son opcionales.** Si no, MySQL falla al agregarlos sobre filas existentes. `Order` no tiene lista de detalles y no debe tenerla.
15. **Tests con Mockito.** `MockitoExtension` es estricto: un *stub* que no se usa falla el test. `@InjectMocks` `OrderBusiness` inyecta por tipo: el campo nuevo `OrderDetailRepository` se mockea con `@Mock`.
16. **Postman.** Las carpetas 05 y 06 usan variables propias. No reutilices las de las carpetas 00 a 04 (`orderNumber`, `orderNumberLoad`, `orderNumberClose`, `activationPassword`…): rompen sus aserciones. El token dura 1 hora: la carpeta 06 hace su propio login.
17. **`FinalWeightRequest` solo tiene `weight`.** No hay fecha del cliente: la estampa es del servidor (consigna: "momento de recepción").
18. **Números como texto y rutas no numéricas.** Un `weight` `"abc"` o un `orderNumber` `abc` dan 400 con el formato por defecto de Spring (H-21 y H-23); un número como texto (`"16490"`) se acepta. Se documenta, no se cambia.
19. **H-25 y D14.** El bloqueo en `activate` va **después** de comprobar el cuerpo nulo (si no, se consultaría la base con un `null`) y no cambia el orden 400 → 404 → 409 → 403.

5. Fuera de alcance de la Fase 5

- **Examen final:** roles por endpoint, alarma de temperatura por mail (eventos de la clase 9), frontend y endurecimiento de seguridad (H-5).
- **Opcional del parcial:** frecuencia de almacenamiento distinta de la de recepción (se siguen guardando todas las lecturas válidas).

- **H-12** (taras simultáneas sobre la misma orden): sigue aceptado; no se cambia `registerTare` más que lo de D13.
- Un endpoint que liste conciliaciones, paginación de conciliaciones, reabrir una orden y campos nuevos para valores calculados.
- Cambiar la decisión 30 (exigir una lectura al cerrar): es un cambio de la Fase 4 y no hace falta con D6.

6. Criterios de aceptación de la fase

#	Criterio
1	Pesaje final válido (orden en estado 3): 200 con la conciliación y la orden queda en **estado 4** (también en `GET /orders/{n}`)
2	`finalWeight` y `finalWeighingDateTime` guardados y visibles en `GET /orders/{n}` (la fecha es del servidor)
3	El historial suma una fila **3 → 4** con el usuario del token y la observación (el circuito completo deja 4 filas: `null→1`, `1→2`, `2→3`, `3→4`)
4	Las fórmulas coinciden con la sección 3.5: `loadedProduct` 2000.0, `scaleNet` 1990.0, `difference` −10.0
5	Los promedios son el `AVG` de las lecturas guardadas (21.0, 0.5 y 3300.0 en el ejemplo)
6	Orden cerrada sin lecturas: 200, `loadedProduct` 0.0 y promedios `null` (D6)
7	Peso ausente, 0, negativo o infinito: 400. Peso menor que la tara: 400. Peso igual a la tara: 200
8	Un 400 no cambia nada: la orden sigue en estado 3 con `finalWeight` nulo y sin fila nueva en el historial
9	Estados 1, 2 y 4: 409 · segundo pesaje final: 409 · orden inexistente: 404 · sin token: 401
10	`GET .../reconciliation` en estado 4: 200 con los **mismos valores** que devolvió el POST · estados 1, 2 y 3: 409 · inexistente: 404 · sin token: 401
11	Con la orden en estado 4, lecturas, activación, cierre, tara y pesaje final dan 409 sin cambiar su código
12	Concurrencia: dos pesajes finales simultáneos dan un 200 y un 409; la orden queda en estado 4 con **una sola** fila 3 → 4
13	**H-26:** `POST .../tare` con `1e999` responde 400 (no 500) y la orden queda en estado 1 con tara nula
14	**H-25:** una activación y un cierre simultáneos nunca dejan la orden en estado 2 después de que el cierre respondió 200; dos activaciones simultáneas guardan `activationDateTime` una sola vez
15	Las columnas nuevas de `orders` se agregan sobre una base con órdenes existentes, sin error
16	Swagger documenta los dos endpoints con sus respuestas
17	Postman: las carpetas 05 y 06 pasan con newman y las carpetas 00 a 04 siguen pasando (86 pedidos y 204 aserciones)
18	Tests unitarios nuevos y `./mvnw -B test` en verde (los 19 previos más los nuevos)
19	La carpeta 06 simula el circuito 1 → 4 encadenado, con *Tests* de Postman
20	Auditoría final: los Puntos 1 a 5 y los requisitos obligatorios del segundo parcial verificados contra el PDF oficial
21	Informe y guía de la Fase 5 (≥ 10 preguntas), `CLAUDE.md`, plan y guía de `src` actualizados, y PDF

7. Reparto y propiedad de archivos

Como el trabajo es **secuencial**, no hay riesgo de que dos personas editen a la vez. Aun así, cada archivo tiene un dueño y los que tocan dos partes se **extienden sin reescribir** lo anterior.

Archivo	Acción	Dueño
`model/Order.java`	Modificar (2 campos)	**Parte 1**
`model/persistence/OrderDetailRepository.java`, `model/persistence/ReadingAverages.java`	Agregar la consulta / crear la proyección	**Parte 1**
`business/Reconciliation.java`	Crear	**Parte 1**
`business/IOrderBusiness.java`, `business/OrderBusiness.java`	Agregar `registerFinalWeight`, `reconciliation`, `requireValidWeight` y `buildReconciliation`; **arreglos** de `registerTare` (D13) y `activate` (D14)	**Parte 1**

Archivo	Acción	Dueño
`integration/tms/model/FinalWeightRequest.java`	Crear	**Parte 1**
`integration/tms/controllers/OrderTmsRestController.java`	**Agregar** el método (sin tocar `registerTare`)	**Parte 1**
`controllers/OrderRestController.java`	**Agregar** el método (sin tocar los otros)	**Parte 1**
Tests nuevos en `src/test/java/ar/edu/iua/iw3/business/`	Crear	**Parte 1**
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Agregar las carpetas 05 y 06	**Parte 2**
`docs/fase-5-conciliacion.md` y su PDF	Crear	**Parte 2**
`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md`	Actualizar al cerrar	**Parte 2**
Este documento (bloque "Estado de la Parte N") y filas de `docs/guia-estudio-src.md` para sus archivos	Agregar	**Cada parte**, lo suyo

No se commitean las carpetas `.postman/` ni `postman/` (son el vínculo con el espacio de trabajo de Postman de cada uno).

Estimación (a ajustar): Parte 1 ≈ 5 a 6 h · Parte 2 ≈ 5 a 6 h (la más larga por la escritura de las dos carpetas, la concurrencia y la auditoría; se alivia porque la Parte 1 deja su bloque de estado y sus filas de la guía listos).

8. Orden de trabajo y traspasos

```
Etapa 0   Martín publica la rama y este documento                    ← Calvo espera este push
Parte 1   Calvo implementa, verifica, pushea y AVISA a Alasino
Parte 2   Alasino trae la rama, hace Postman + concurrencia + auditoría + documentación y abre el PR (borrador)
Cierre    Revisión del PR: Martín, Contreras y Calvo aprueban → merge
```

Propuesta de fechas (a ajustar entre ustedes): Parte 1 de martes 3/11 a miércoles 4/11 · Parte 2 de jueves 5/11 a viernes 6/11 · PR y aprobaciones de sábado 7/11 a lunes 9/11 · **entrega martes 10/11**. La Fase 4 ya está en `main` (PR #12, 31/10).

Cada parte, al terminar, hace tres cosas antes de pasar la posta:

1. Agrega al final de este documento el bloque **"Estado de la Parte N"** (modelo en el Anexo B): qué quedó hecho, commits, qué se verificó con resultados reales, qué debe tener en cuenta el siguiente.
2. Agrega a `docs/guia-estudio-src.md` las filas de sus archivos nuevos, marcadas como `F5`.
3. Pushea y le avisa al siguiente.

Mientras esperás tu turno podés preparar sin pushear: Alasino puede diseñar los casos de las carpetas 05 y 06, el esqueleto de la matriz de auditoría y el script de concurrencia.

Revisión cruzada (para la evaluación individual): Alasino audita lo que hizo Calvo (apartado de auditoría de la Parte 2) y Calvo revisa el PR de Alasino. **Martín** y **Contreras**, que no programan en esta fase, revisan el PR completo y lo aprueban: junto con Calvo suman las 3 aprobaciones del acuerdo (decisión 14; el autor del PR, Alasino, no cuenta). Hoy el *ruleset* de `main` está deshabilitado: GitHub no lo exige, así que el acuerdo se cumple a mano.

9. Cómo trabajar con Git y cómo probar

```
# Solo Martín, una vez (la rama ya está creada en su máquina desde main actualizado):
git checkout fase-5-conciliacion
# ... commits: (a) este .md, (b) su PDF, (c) docs/pdf/README.md ...
git push -u origin fase-5-conciliacion

# Calvo y Alasino, cuando el anterior avise que publicó:
git fetch
git checkout fase-5-conciliacion
git pull --rebase

# Todos, antes de CADA push:
git pull --rebase
git push
```

Regla	Detalle
Commits	**Atómicos**, de **100 líneas o menos**, en español, formato `tipo(ámbito): descripción` (`feat`, `fix`, `docs`, `test`, `build`, `refactor`). Están **autorizados los commits locales**
Excepción de las 100 líneas	Documentos nuevos completos y el JSON de Postman: un commit aparte por archivo
Coautoría	**Sin** `Co-Authored-By: Claude` ni "Generated with Claude Code"
Push	Solo cuando el responsable lo pide, y siempre después de `git pull --rebase`
`main`	Nadie pushea a `main`. Entra por Pull Request
Force push	Prohibido
Alcance	No agregar dependencias, no refactorizar fuera de la fase y no tocar las Fases 1 a 4 más que lo de D13 y D14

Si el push es rechazado ("non-fast-forward"): alguien pusheó antes. `git pull --rebase` y volver a pushear. **Si hay un conflicto**, alguien editó un archivo ajeno: avisen y decidan entre los dos dueños; no lo resuelvan a ciegas.

Probar en tu máquina (recomendado: una base descartable para no ensuciar `iw3_db`):

```
docker compose up -d                 # la base (contenedor iw3-gas-mysql)
docker exec iw3-gas-mysql mysql -uroot -proot -e "create database iw3_scratch"
./mvnw spring-boot:run "-Dspring-boot.run.arguments=--server.port=8089 --
spring.datasource.url=jdbc:mysql://localhost:3306/iw3_scratch?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true"
# Con la app ya iniciada, sembrar el usuario de prueba (PowerShell):
Get-Content docs/sql/seed-dev-users.sql | docker exec -i iw3-gas-mysql mysql -uroot -proot iw3_scratch
# Al terminar: bajar la app y  docker exec iw3-gas-mysql mysql -uroot -proot -e "drop database iw3_scratch"
```

Usuario de desarrollo: `admin` / `admin123`. Swagger: `http://localhost:8089/swagger-ui.html` (o 8080 si usás la base normal).

```
TOKEN=$(curl -s -X POST localhost:8089/api/v1/login -d "username=admin&password=admin123")

# Orden en estado 3 (crear en SAP, tara en TMS, activar, lecturas y cerrar: carpetas 01 a 04 de Postman, o
Swagger)

# Pesaje final
curl -i -X POST localhost:8089/api/v1/integration/tms/orders/90001/final-weight \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"weight": 16490.0}'

# Conciliación
curl -i localhost:8089/api/v1/orders/90001/reconciliation -H "Authorization: Bearer $TOKEN"

# Mirar la base
docker exec iw3-gas-mysql mysql -uroot -proot iw3_scratch -e "SELECT order_number, status, tare, final_weight,
final_weighing_date_time, last_accumulated_mass FROM orders ORDER BY id DESC LIMIT 5;"
docker exec iw3-gas-mysql mysql -uroot -proot iw3_scratch -e "SELECT id_order, from_status, to_status, date_time,
actor, observations FROM order_status_logs ORDER BY id DESC LIMIT 8;"
```

En PowerShell usá `curl.exe` y comillas simples para el JSON, o directamente Swagger.

Parte 1 — Negocio y endpoints (pesaje final, conciliación y arreglos)

Responsable: Tomás Calvo · **Estimación:** ≈ 5 a 6 h · **Es la primera: la Parte 2 depende de su push.**

Punto de partida (qué heredás de la Fase 4)

`Order` tiene la tara y los últimos valores, pero **no** `finalWeight` ni su fecha. `OrderDetailRepository` solo tiene `findByOrder`. `IOrderBusiness` termina en `close`. `OrderTmsRestController` solo tiene `/tare` y `OrderRestController` no tiene `/reconciliation`. `OrderStatus.FINISHED` existe pero nadie lo usa. `registerTare` rechaza `!(weight > 0)` pero **no** el infinito (H-26), y `activate` busca la orden **sin** bloqueo (H-25).

Objetivo

Recibir el pesaje final (estado 3 → 4), calcular y devolver la conciliación, permitir consultarla en el estado 4, y corregir los dos defectos heredados (H-26 y H-25). La Parte 2 se apoya en lo que dejes.

Archivos

Todos bajo `src/main/java/ar/edu/iua/iw3/` salvo los tests.

Archivo	Acción
`model/Order.java`	Agregar `finalWeight` y `finalWeighingDateTime` (sección 3.1)
`model/persistence/ReadingAverages.java`	Crear (interfaz de proyección)
`model/persistence/OrderDetailRepository.java`	Agregar `averagesByOrder`
`business/Reconciliation.java`	Crear (sección 3.3)
`business/IOrderBusiness.java` y `OrderBusiness.java`	Agregar `registerFinalWeight` y `reconciliation`; helpers privados `requireValidWeight` y `buildReconciliation`; **arreglar** `registerTare` (D13) y `activate` (D14)

Archivo	Acción
`integration/tms/model/FinalWeightRequest.java`	Crear
`integration/tms/controllers/OrderTmsRestController.java`	**Agregar** el método del pesaje final (sin tocar `registerTare`)
`controllers/OrderRestController.java`	**Agregar** el método de la conciliación (sin tocar los otros)
`src/test/java/ar/edu/iua/iw3/business/OrderBusinessFinalWeightTest.java`	Crear (Mockito)
`src/test/java/ar/edu/iua/iw3/business/OrderBusinessTareTest.java`	Crear (Mockito, para H-26)
`src/test/java/ar/edu/iua/iw3/business/OrderBusinessActivateTest.java`	Crear (Mockito, para H-25)

Qué hacer, paso a paso

Paso 1 — `Order`. Dos campos opcionales con `@Schema(accessMode = READ_ONLY)` y un comentario breve cada uno.

Paso 2 — Promedios. `ReadingAverages` (interfaz con los tres getters `Double`) y `averagesByOrder` en `OrderDetailRepository` con la consulta de la sección 3.1.

Paso 3 — `Reconciliation`. Clase con los nueve campos, `@Getter`, `@AllArgsConstructor` y `@Schema` con ejemplos (los de la sección 3.5).

Paso 4 — Arreglo H-26 (D13). Helper privado `requireValidWeight(double weight) throws BadRequestException` en `OrderBusiness`: `!(weight > 0)` → 400 con el texto actual `weight debe ser mayor que 0 (kg)`; `Double.isInfinite(weight)` → 400 con `weight debe ser un número finito`. Usalo en `registerTare` en lugar de la comprobación actual (el resto de `registerTare` no cambia). Test `OrderBusinessTareTest`.

Paso 5 — Arreglo H-25 (D14). En `activate`, después de la comprobación de `orderNumber` / `password` nulos, reemplazá `load(orderNumber)` por `loadForUpdate(orderNumber)`. El orden 400 → 404 → 409 → 403 no cambia. Test `OrderBusinessActivateTest`.

Paso 6 — Interfaz. `registerFinalWeight` y `reconciliation` en `IOrderBusiness`, con Javadoc en español (qué comprueba, qué cambia, qué escribe).

Paso 7 — `registerFinalWeight`. Las reglas de la sección 3.4, en ese orden, con `@Transactional(rollbackFor = Exception.class)`, las cinco primeras comprobaciones **fuera del** `try` y `saveAndFlush` adentro. Inyectá `OrderDetailRepository` en `OrderBusiness` (no `IOrderDetailBusiness`).

Paso 8 — `reconciliation` y `buildReconciliation`. La consulta con las reglas de la sección 3.4 y la función compartida, que usan el POST y el GET.

Paso 9 — DTO y endpoints. `FinalWeightRequest` con `double weight` primitivo y `@Schema`. En `OrderTmsRestController`: `POST /{orderNumber}/final-weight` (consumes y produces JSON), con `@Operation` y `@ApiResponse` (200, 400, 404, 409), `getUserLogged()` para el actor y la traducción de errores con `IStandartResponseBusiness`: 404, 400, 409 y 500. En `OrderRestController`: `GET /{orderNumber}/reconciliation` con 200, 404, 409 y 500.

Paso 10 — Cierre de la parte: bloque "Estado de la Parte 1" y filas de la guía de `src`.

Plan de commits sugerido (todos de menos de 100 líneas):

1. `feat(model): agrega a la orden el pesaje final y su fecha`
2. `feat(model): agrega la consulta de promedios del detalle de la orden`
3. `feat(business): agrega la clase de la conciliación`
4. `fix(business): rechaza pesos infinitos en la tara (H-26)`

5. `test(business): prueba la tara con pesos inválidos`
6. `fix(business): busca la orden con bloqueo en la activación (H-25)`
7. `test(business): prueba la activación con bloqueo`
8. `feat(business): declara el pesaje final y la conciliación en la interfaz`
9. `feat(business): implementa el pesaje final de la orden`
10. `feat(business): implementa la consulta de la conciliación`
11. `test(business): prueba el pesaje final y la conciliación`
12. `feat(integration): agrega el pedido del pesaje final de la balanza`
13. `feat(integration): agrega el endpoint del pesaje final`
14. `feat(controllers): agrega la consulta de la conciliación`
15. `docs(fase-5): registra el estado de la Parte 1 y actualiza el mapa de código`

Cómo probar

Primero necesitás una orden en estado 3: creala (SAP), registrá la tara (TMS), activala, mandá dos lecturas y cerrala (carpetas 01 a 04 de Postman, Swagger o `curl`; los valores del ejemplo de la sección 3.5).

Caso	Esperado
`{}`, peso 0, peso negativo	400 (`weight debe ser mayor que 0`); la orden sigue en estado 3
Peso `1e999` en el pesaje final	400 (`weight debe ser un número finito`)
Peso `1e999` en la **tara** (orden en estado 1)	400, y la orden queda en estado 1 con tara nula (H-26)
Peso menor que la tara (14000)	400 (`no puede ser menor que la tara`)
Orden inexistente	404
Orden en estado 1 o 2	409 (`no admite pesaje final`)
Peso `16490.0`	200 con la conciliación del ejemplo; orden en estado 4
El mismo pesaje otra vez	409
`GET /orders/{n}`	`status` 4, `finalWeight` y `finalWeighingDateTime` presentes
`GET /orders/{n}/status-log`	4 registros; el último `3→4`, con el usuario y `Pesaje final registrado`
`GET .../reconciliation` en estado 4	200, mismos valores que el POST
`GET .../reconciliation` en estado 1, 2 o 3	409 · inexistente 404
Orden cerrada **sin lecturas**, peso igual a la tara	200, `loadedProduct` 0.0, `scaleNet` 0.0, `difference` 0.0 y promedios `null`
Sin token	401
Lectura, activación, cierre y tara sobre la orden en estado 4	409 en todas (sin tocar su código)

También comprobá:

- **Los promedios con cero lecturas** devuelven `null` y no una excepción (sección 3.1, "a verificar").
- **H-25:** `activate` pide la orden con bloqueo (lo comprueba el test) y la activación sigue comportándose igual (400 → 404 → 409 → 403; idempotente).
- **La app arranca sobre una base con órdenes**, y Hibernate agrega las dos columnas nuevas (opcionales).
- **Regresión:** `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json` sigue dando **86 pedidos y 204 aserciones** sin fallas, y `./mvnw -B test` los 19 tests previos más los nuevos.
- **Swagger** (`/v3/api-docs`) muestra los dos endpoints con sus respuestas.
- `./mvnw -B -q -DskipTests compile` sin errores tras cada commit de código.

Criterio de terminado

- Los casos de la tabla se cumplen y las comprobaciones de arriba también.
- Tests unitarios nuevos en verde y regresión de Postman sin fallas.
- Commits locales hechos y **publicados a la rama** (cuando lo pidas).
- Bloque "Estado de la Parte 1" escrito, y **Alasino avisado**.

No tocar

Postman, `CLAUDE.md`, el plan y los informes (Parte 2); la frecuencia de almacenamiento; `Constants`; `OrderDetailBusiness` y su `loadForUpdate`; el resto de `registerTare` y de `activate`. No implementes roles, alarmas ni nada del frontend. Si necesitás un cambio ajeno, avisá.

Qué le dejás a Alasino

Los dos endpoints funcionando, la lista de casos que ya probaste a mano (para convertirlos en pedidos de Postman), los valores exactos de la conciliación del ejemplo, el resultado de la verificación de promedios con cero lecturas y los dos arreglos (H-25 y H-26) con sus tests.

Prompt para Claude Code (Parte 1)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 5 (punto 5: pesaje final y conciliación), rama fase-5-conciliacion. El trabajo es SECUENCIAL: primero Calvo (esta parte) y después Alasino. Martín González autorizó el planeamiento de la fase el 02/11/2026 y confirmó las decisiones D1 a D14 del documento de división, los commits locales atómicos y los dos arreglos de las Fases 2 y 3 (D13: H-26 y D14: H-25). La Fase 4 ya está en main (PR #12). Algunas líneas de CLAUDE.md (por ejemplo que la Fase 4 espera su PR) quedaron desactualizadas: no las edites; las corrige Alasino al cerrar la fase.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-5-division-de-trabajo.md (secciones 1 a 6 y "Parte 1"), y docs/pdf/consigna-trabajo-integrador-2026.pdf (página 3: Punto 5; páginas 4 y 5: anexo de la orden). Estudiá también business/OrderBusiness.java (registerTare, activate, close, loadForUpdate y logStatus), business/IOrderBusiness.java, model/Order.java, model/persistence/OrderRepository.java y OrderDetailRepository.java, integration/tms/controllers/OrderTmsRestController.java, integration/tms/model/TareRequest.java, controllers/OrderRestController.java y src/test/java/ar/edu/iua/iw3/business/OrderBusinessCloseTest.java (molde de los tests).

Soy Tomás Calvo y hago la PARTE 1 (negocio y endpoints). Antes de empezar hacé git fetch, git checkout fase-5-conciliacion y git pull --rebase, y comprobá que Martín publicó la rama con docs/fase-5-division-de-trabajo.md. Si falta, avisame y esperá.

Implementá SOLO estos archivos (rutas bajo src/main/java/ar/edu/iua/iw3/ salvo los tests):
1. model/Order.java: agregar finalWeight (Double) y finalWeighingDateTime (LocalDateTime), opcionales (la tabla orders ya tiene filas), con @Schema(accessMode = READ_ONLY) y un comentario breve. NO agregues ninguna lista de detalles.
2. model/persistence/ReadingAverages.java: interfaz de proyección con Double getTemperature(), Double getDensity() y Double getFlowRate(). En model/persistence/OrderDetailRepository.java agregá UNA consulta: @Query("select avg(d.temperature) as temperature, avg(d.density) as density, avg(d.flowRate) as flowRate from OrderDetail d where d.order = ?1") ReadingAverages averagesByOrder(Order order). Verificá que con cero lecturas devuelve promedios nulos y no una excepción; si devolviera null, tratalo igual (promedios null).
3. business/Reconciliation.java: clase pública con @Getter y @AllArgsConstructor (Lombok) y @Schema con ejemplos: Integer orderNumber, Double tare, Double finalWeight, Double loadedProduct, Double scaleNet, Double difference, Double averageTemperature, Double averageDensity, Double averageFlowRate. Es el cuerpo de respuesta del POST y del GET.
4. business/OrderBusiness.java, arreglo H-26 (D13): helper privado requireValidWeight(double weight) throws BadRequestException: !(weight > 0) -> 400 con el texto ACTUAL "weight debe ser mayor que 0 (kg)" (no lo cambies: la carpeta 02 de Postman lo comprueba); Double.isInfinite(weight) -> 400 con "weight debe ser un número finito". Usalo en registerTare en lugar de su comprobación actual de peso; el resto de registerTare no cambia.
5. business/OrderBusiness.java, arreglo H-25 (D14): en activate, DESPUÉS de la comprobación de orderNumber y password nulos, reemplazá load(orderNumber) por loadForUpdate(orderNumber). El orden 400 -> 404 -> 409 -> 403, el mensaje del 403 (sin contraseñas) y la idempotencia no cambian.
6. business/IOrderBusiness.java y OrderBusiness.java: agregar (a) Reconciliation registerFinalWeight(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException y (b) Reconciliation reconciliation(Integer orderNumber) throws NotFoundException, ConflictException, BusinessException, con Javadoc en español. OrderBusiness inyecta OrderDetailRepository DIRECTO (no IOrderDetailBusiness: OrderDetailBusiness ya depende de IOrderBusiness y sería una dependencia circular).
7. registerFinalWeight con @Transactional(rollbackFor = Exception.class) y estas comprobaciones FUERA del try, en este orden: (a) loadForUpdate(orderNumber) -> 404 (reutilizá el helper privado que ya existe; NO crees otro duplicado); (b) requireValidWeight(weight) -> 400; (c) estado distinto de CLOSED_FOR_LOADING -> 409 con "La orden <n> no admite pesaje final: debe estar en estado 3 (estado <e>)"; (d) tare nula -> 409 con "La orden <n> no tiene tara registrada"; (e) weight < tare -> 400 con "weight (pesaje final) no puede ser menor que la tara (<t> kg)" (un peso IGUAL a la tara es válido). Dentro del try: from = estado actual; setFinalWeight(weight); setFinalWeighingDateTime(LocalDateTime.now()); estado FINISHED; saveAndFlush(order); logStatus(order, from, FINISHED, actor, "Pesaje final registrado"); devolver buildReconciliation(order). Cualquier otra falla -> log.error y BusinessException.
8. reconciliation: load(orderNumber) -> 404 (fuera del try); estado distinto de FINISHED -> 409 con "La conciliación solo está disponible para órdenes en estado 4 (orden <n> en estado <e>)"; devolver buildReconciliation(order) con el error técnico envuelto en BusinessException. Sin bloqueo.
9. buildReconciliation(Order) privado, UNA sola función para el POST y el GET: tare = order.getTare(); finalWeight = order.getFinalWeight(); loadedProduct = lastAccumulatedMass o 0.0 si es nulo; scaleNet = finalWeight - tare; difference = scaleNet - loadedProduct; promedios = orderDetailDAO.averagesByOrder(order), con null si no hay lecturas. SIN redondeo. La conciliación NO se guarda.
10. integration/tms/model/FinalWeightRequest.java: double weight PRIMITIVO (como TareRequest: un campo ausente llega como 0.0 y da 400), con @Schema y ejemplo 16490.0. Sin fecha: la estampa es del servidor.
11. integration/tms/controllers/OrderTmsRestController.java: AGREGÁ POST /{orderNumber}/final-weight (consumes y produces JSON) con @Operation y @ApiResponse (200 con el esquema Reconciliation, 400, 404, 409). Actor con getUserLogged() como registerTare. Traducción con IStandartResponseBusiness.build(status, e, e.getMessage()): NotFoundException -> 404, BadRequestException -> 400, ConflictException -> 409, BusinessException -> 500. NO
```

```
modifiques registerTare.
12. controllers/OrderRestController.java: AGREGÁ GET /{orderNumber}/reconciliation (produces JSON) con @Operation
y @ApiResponse (200, 404, 409): NotFoundException -> 404, ConflictException -> 409, BusinessException -> 500. NO
modifiques los otros métodos. No hace falta tocar Constants.
13. Tests en src/test/java/ar/edu/iua/iw3/business/ con Mockito (@ExtendWith(MockitoExtension.class),
@InjectMocks OrderBusiness, @Mock OrderRepository, OrderStatusLogRepository y OrderDetailRepository; cuidado:
Mockito es estricto con los stubs sin usar). OrderBusinessFinalWeightTest: orden inexistente (404); peso 0,
negativo e infinito (400, sin tocar el repositorio); estados 1, 2 y 4 (409); tara nula (409); peso menor que la
tara (400); peso igual a la tara (válido); caso feliz (estado 4, finalWeight y fecha guardados, UNA fila de log
3->4 con el actor y la observación; actor nulo o en blanco -> "sistema"); conciliación con los valores del
ejemplo (tara 14500.0, lastAccumulatedMass 2000.0, peso 16490.0, promedios 21.0, 0.5 y 3300.0 -> loadedProduct
2000.0, scaleNet 1990.0, difference -10.0); sin lecturas (loadedProduct 0.0 y promedios null); reconciliation:
404, estados 1, 2 y 3 (409) y estado 4 (valores). OrderBusinessTareTest: peso infinito -> BadRequestException sin
guardar; "weight debe ser mayor que 0" se mantiene para 0, negativo y NaN; una tara válida sigue pasando a estado
2. OrderBusinessActivateTest: usa findByOrderNumberForUpdate (verificalo); orden inexistente 404; estado distinto
de 2 -> 409; contraseña incorrecta -> 403; primera activación guarda activationDateTime; segunda no lo cambia.

NO toques: Postman (ni el JSON ni .postman/ ni postman/), CLAUDE.md, docs/plan-de-trabajo.md ni los informes
(Parte 2), Constants, OrderDetailBusiness, el resto de registerTare y de activate, ni el modelo más allá de los
dos campos. No implementes la frecuencia de almacenamiento, roles, alarmas ni frontend. No agregues dependencias
ni refactorices fuera de la fase. Si creés que hace falta un cambio ajeno, avisame en vez de hacerlo.

Reglas: metodología de la cátedra (capas, I*Business, excepciones checked, StandartResponse); identificadores en
inglés y comentarios en español. Todas las comprobaciones fuera del try y saveAndFlush dentro. Commits atómicos
de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated
with Claude Code" (están autorizados los commits locales). Plan de commits sugerido: modelo; promedios;
Reconciliation; fix H-26; test de la tara; fix H-25; test de activate; interfaz; pesaje final; conciliación;
tests del pesaje final; FinalWeightRequest; endpoint del pesaje final; endpoint de la conciliación; docs. No
hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. Antes de editar,
mostrame tu plan de archivos y commits y esperá mi OK.

Verificá de verdad y reportá solo lo que ejecutaste: compilá (./mvnw -B -q -DskipTests compile) tras cada commit
de código; corré ./mvnw -B test (esperado: los 19 tests previos más los nuevos, 0 fallas). Con Docker en marcha
usá una base descartable (docker exec iw3-gas-mysql mysql -uroot -proot -e "create database iw3_scratch"; arrancá
la app en otro puerto con ./mvnw spring-boot:run y -Dspring-boot.run.arguments entre comillas con --
server.port=8089 y --spring.datasource.url igual a la de application.properties pero con la base iw3_scratch;
sembrá el usuario con docs/sql/seed-dev-users.sql). Primero arrancá sobre una base que ya tenga órdenes y
confirmá que Hibernate agrega final_weight y final_weight_date_time sin tocar los datos. Armá una orden en estado
3 (crear, tara 14500, activar, lecturas masa 1000/densidad 0.5/temperatura 20/caudal 3000 y masa
2000/0.5/22/3600, cerrar) y probá: {} y peso 0 y negativo (400); peso 1e999 en el pesaje final (400); peso 1e999
en la tara de una orden en estado 1 (400 y la orden sigue en estado 1 con tara nula); peso 14000 (400); orden
inexistente (404); orden en estado 1 y en estado 2 (409); peso 16490.0 (200 con loadedProduct 2000.0, scaleNet
1990.0, difference -10.0, averageTemperature 21.0, averageDensity 0.5, averageFlowRate 3300.0 y la orden en
estado 4); repetir el pesaje (409); GET /orders/{n} (status 4, finalWeight y finalWeighingDateTime); /status-log
(4 registros, el último 3->4 con el usuario y "Pesaje final registrado"); GET /reconciliation (200 con los mismos
valores) y en estados 1, 2 y 3 (409) e inexistente (404); una orden cerrada SIN lecturas con peso igual a la tara
(200, loadedProduct 0.0, scaleNet 0.0, difference 0.0 y promedios null); sin token (401); lecturas, activación,
cierre y tara sobre la orden en estado 4 (409 todas); y la activación (400, 404, 409, 403, idempotencia).
Comprobá con SELECT que solo hay una fila 3->4 y que los 400 no escribieron nada. Corré npx newman@6 run
docs/postman/IW3-Gas-Terminal.postman_collection.json (tienen que seguir pasando los 86 pedidos y las 204
aserciones, 0 fallas) y confirmá en /v3/api-docs que los dos endpoints figuran con sus respuestas. Al terminar
bajá la app y borrá la base descartable. Si algo falla, decilo.

Al terminar: agregá al final de docs/fase-5-division-de-trabajo.md el bloque "Estado de la Parte 1" (qué quedó
hecho, commits, qué verificaste con resultados reales, los casos que Alasino debe pasar a Postman, el resultado
de promedios con cero lecturas y qué debe tener en cuenta) y las filas de tus archivos en docs/guia-estudio-
src.md marcadas F5 (commits aparte). No pushees hasta que yo lo pida.
```

Parte 2 — Postman, concurrencia, auditoría final, documentación y PR

Responsable: Benjamín Alasino · **Estimación:** ≈ 5 a 6 h · **Cierra la fase y cierra el parcial.**

Punto de partida (qué heredás de la Parte 1)

Antes de empezar, traé la rama y comprobá que están los dos endpoints funcionando (`final-weight` y `reconciliation`), los arreglos de H-26 y H-25, `Order` con los dos campos nuevos y los tests de la Parte 1. Leé el bloque "Estado de la Parte 1" (con los casos probados y los valores de la conciliación). Las carpetas 00 a 04 de Postman (86 pedidos, 204 aserciones) son tu punto de comparación para las regresiones.

Objetivo

Probar la fase de punta a punta, **simular el circuito completo 1 → 4**, **auditar si el segundo parcial está completo contra el PDF oficial**, documentar la fase, actualizar la documentación del proyecto y abrir el PR.

Qué hacer, paso a paso

Apartado A — Postman: carpeta `05 - Fase 5: pesaje final y conciliación (TMS)`

Variables **propias** (por ejemplo `orderNumberFinal`, `activationPasswordFinal`, `orderNumberFinalPending`, `orderNumberFinalBare`, `orderNumberFinalTare`). Estilo y generación del número de orden como en las carpetas 02 a 04 (`Date.now()` en el *pre-request script*). **No uses** variables de las carpetas 00 a 04.

Grupo	Pedidos (cada uno con sus tests)	Esperado
Preparación	Crear la orden · Tara 14500 (guarda la contraseña) · Activar · Lectura 1 y 2 (los valores de la sección 3.5) · Cerrar	201 · 200 · 200 · 200 `saved: true` ×2 · 200 estado 3
Pesaje final inválido	Sin campo (`{}`) · Peso 0 · Peso negativo · Peso infinito (`1e999`) · Peso menor que la tara (14000) · Consultar la orden	400 ×5 con fragmentos (`mayor que 0`, `finito`, `menor que la tara`) · la orden sigue en estado 3, `finalWeight` nulo
Errores	Orden inexistente · Sin token	404 · 401
Pesaje final	Peso 16490.0 · Consultar la orden · Historial · Conciliación (GET)	200 con la conciliación del ejemplo (comparar con `closeTo`) · estado 4 con `finalWeight` y `finalWeighingDateTime` · **4** registros, el último `3→4`, con el usuario `admin` y `Pesaje final registrado` · mismos valores que el POST
Estado 4 es final	Segundo pesaje final · Lectura · Activación · Cierre · Tara	409 en todos (fragmentos `no admite pesaje final`, `no acepta lecturas`, `no está lista para activar`, `no se puede cerrar`, `no está pendiente`)
Conciliación fuera del estado 4	GET en una orden en estado 1 · en estado 2 · en estado 3 · inexistente · sin token	409 ×3 (fragmento `solo está disponible para órdenes en estado 4`) · 404 · 401
Pesaje final fuera del estado 3	POST en una orden en estado 1 · en estado 2	409 ×2
Límite (D2 y D6)	Orden sin lecturas: crear, tara, cerrar, pesaje final igual a la tara	200 con `loadedProduct` 0.0, `scaleNet` 0.0, `difference` 0.0 y promedios `null`
Regresión H-26	Tara con `1e999` en una orden en estado 1 · consultar la orden	400 (fragmento `finito`) · estado 1 y `tare` nula

Las aserciones sobre mensajes usan **fragmentos** (`include`), según la sección 3.6. Los valores con residuos posibles se comparan con `closeTo`.

Apartado B — Postman: carpeta `06 - Circuito completo (1 a 4)`

Un recorrido **encadenado** de una orden de punta a punta, con *Tests* de Postman y variables propias (`orderNumberCircuit`, `activationPasswordCircuit`, …), independiente de las otras carpetas. Hace su **propio login**

(el token dura 1 hora). Cada pedido guarda en variables lo que usa el siguiente.

1. Login · **Punto 1:** crear la orden (201, estado 1) y reenviarla (302).
2. **Punto 2:** tara (200, guarda la contraseña) · la orden pasa a estado 2.
3. **Punto 3:** activación (200 con el preset) · cuatro lecturas válidas y una descartada (`saved: false`) · la cabecera guarda los últimos valores.
4. **Punto 4:** cierre (200, estado 3) · una lectura tardía (409).
5. **Punto 5:** pesaje final · la conciliación se **calcula en el test** a partir de los datos enviados y se compara con la respuesta (`closeTo`) · `GET .../reconciliation` coincide con el POST.
6. **Historial:** exactamente 4 registros con la secuencia `null→1`, `1→2`, `2→3`, `3→4` y el usuario en cada uno.
7. La orden aparece en el listado paginado (`GET /orders`) y el histórico de lecturas (`GET /orders/{n}/details`) tiene las lecturas guardadas.

El JSON de Postman va en **un commit aparte** (o uno por carpeta), solo ese archivo, y con `git diff` verificás que **solo cambie lo nuevo** (las carpetas 00 a 04 no deben cambiar).

Apartado C — Pruebas de punta a punta y de concurrencia

```
npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json
./mvnw -B test        # con la base arriba
```

Registrá los comandos y los resultados **reales** (pedidos, aserciones y fallas). Tienen que seguir pasando las carpetas 00 a 04 (86 pedidos y 204 aserciones). Pruebas de concurrencia con un script **descartable fuera del repo** y una base descartable:

Prueba	Esperado
Dos pesajes finales simultáneos sobre una orden en estado 3	Un 200 y un 409; la orden en estado 4 con **una sola** fila `3→4`
Una activación y un cierre simultáneos sobre una orden en estado 2 sin activar (H-25), repetido varias veces	Nunca queda en estado 2 tras un cierre 200: resultados posibles `(cierre 200, activación 409)` o `(activación 200, cierre 200)` con estado final 3
Dos activaciones simultáneas de la misma orden (H-16)	Ambas 200 y `activationDateTime` guardado **una sola vez**
Regresión de la Fase 4: unas 50 lecturas y un cierre en paralelo	Sin 500, una sola fila `2→3` y cabecera = máximo guardado

Apartado D — Auditoría final: ¿quedó completo el segundo parcial?

Esta tarea es **adicional** y es la más importante de tu parte. Armá una **matriz de trazabilidad** en el informe: cada requisito, dónde está implementado (archivo y método), con qué prueba se verificó y el resultado. Clasificá cada fila como **implementado y verificado**, **implementado y no verificado**, **parcial**, **con defectos** o **pendiente**.

Revisá como mínimo:

1. **Los Puntos 1 a 5 de la consigna original en PDF** (`docs/pdf/consigna-trabajo-integrador-2026.pdf`), no solo el resumen de `CLAUDE.md`: nombres de campos, reglas, estados y la tabla de la conciliación.
2. **Los requisitos obligatorios del segundo parcial** (PDF, p. 8): APIs de los puntos 1 a 5 **sin interfaz gráfica**; documentación **OpenAPI/Swagger** completa (todos los endpoints de `/v3/api-docs` con `@Operation` y `@ApiResponse`); colección **Postman que pruebe todas las APIs** (cruzá la lista de endpoints de `/v3/api-docs` contra los pedidos de la colección: ¿hay alguno sin cubrir, como `GET /orders` o `/demo/encodepass`?); persistencia, estados secuenciales y unidireccionales, **registro de cada cambio de estado** y descarte de lecturas inválidas.
3. **Los opcionales:** circuito con *Tests* de Postman (carpeta 06), promedios (hechos) y frecuencia de almacenamiento (no hecha, decisión D12).

4. **Los 21 criterios de la sección 6**, uno por uno, con evidencia (pedido, respuesta o consulta SQL).
5. **Las decisiones D1 a D14**: que el código haga exactamente lo decidido (por ejemplo, que el 400 por tara vaya después del 409, que un peso igual a la tara sea válido, que la conciliación no se guarde y que `activate` use el bloqueo).
6. **El orden de las comprobaciones** y que ninguna quede dentro del `try`.
7. **Que no haya lógica duplicada** (una sola función para el POST y el GET), ni dependencias nuevas, ni una lista de detalles en `Order`, ni cambios fuera de lo permitido en las Fases 1 a 4.
8. **Que la contraseña no aparezca** en ningún mensaje, log ni respuesta de consulta.
9. **Regresiones** de las Fases 1 a 4.
10. **El estado 4 es final**: lecturas, activación, cierre, tara y pesaje final dan 409 sin tocar su código.

Si encontrás un defecto: lo registrás con evidencia (pedido exacto, respuesta obtenida, archivo y método) y se lo pasás a su dueño. Si es un cambio mínimo y el dueño lo autoriza, lo corregís vos en un commit `fix(...)` aparte, explicando por qué. **No cierres la fase como completa** si queda un criterio esencial sin cumplir o sin verificar: indicá "parcialmente completa" y qué falta.

Apartado E — Documentación

1. `docs/fase-5-conciliacion.md` con el formato de `docs/fase-4-cierre-orden.md`:
   - **Informe técnico:** alcance y resultado, decisiones D1 a D14 con la alternativa descartada, inventario de archivos por parte (archivo, acción, motivo, responsabilidad), recorrido de las dos peticiones, funciones y métodos, pruebas con resultados reales, **cambios de comportamiento respecto de las fases anteriores** (la tara infinita ahora da 400; la activación bloquea la fila), hallazgos y limitaciones (continuá la numeración: H-26 resuelto y los nuevos desde H-27) y la **matriz de auditoría** del apartado D.
   - **Guía de estudio A–J** con **10 preguntas o más** y respuestas modelo, y una sección "qué debe poder explicar cada integrante" (Calvo y Alasino; los otros dos, lo que revisaron).
   - La atribución es **documental**: distinguí entre responsabilidad asignada y cambios observados en los commits.
2. **Control de calidad:** todo lo documentado coincide con el código real (nombres de clases y métodos, rutas, códigos, ejemplos). Si hay una discrepancia, se corrige el **documento**, no el código.

Apartado F — Actualizar la documentación del proyecto (autorizado por Martín)

Estas correcciones se hacen **al cerrar la Fase 5**, con el estado real ya verificado y con cambios mínimos, en commits aparte:

Documento	Qué actualizar
`CLAUDE.md` sección 2	"Última actualización", "Última fase completada" (Fase 5), "Qué sigue" (PR y entrega), la fila de `main` (hoy dice que recibirá la Fase 4 por PR: ya entró por el PR #12) y la tabla de fases (filas 4 y 5)
`CLAUDE.md` sección 5	La Fase 4 figura como integrada en `main` (PR #12); agregar el bloque de la Fase 5; corregir las cifras de la Fase 3 (carpeta 03: **45 pedidos y 117 aserciones**; hoy dice 44 y 114)
`CLAUDE.md` secciones 6 y 7	Las clases nuevas (`Reconciliation`, `ReadingAverages`, `FinalWeightRequest`); los dos endpoints nuevos con sus respuestas y el ejemplo de la conciliación; `Order` con los dos campos; la cifra de pruebas (tests de Maven y pedidos y aserciones de Postman, medidos al cerrar)
`CLAUDE.md` sección 8.1	Fases 4 y 5 como completadas e integradas / listas para el PR
`CLAUDE.md` sección 8.3	Estado real de la protección de `main`: el *ruleset* existe pero está **deshabilitado** y los PR se mergean con las aprobaciones que haya; no se exige en GitHub
`CLAUDE.md` sección 9	Decisiones 33 a 46

Documento	Qué actualizar
`CLAUDE.md` sección 10	**H-26** resuelto (D13), **H-25** resuelto (D14), **H-16** resuelto como efecto colateral, H-12 sigue aceptado; precisar **H-5**: `dev.info.enabled` es `true` en el perfil por defecto y en `mysqldev` y `false` en `mysqlprod`; `mysqlprod` trae una contraseña por defecto escrita en el `.properties`; el chequeo del *workflow* de deploy apunta a `/api/v1/products` (ruta retirada en la Fase 1; pasa por el 401); y los hallazgos nuevos de tu auditoría
`CLAUDE.md` sección 14	Sumar `docs/fase-4-guia-de-inicio.md`, `docs/fase-4-cierre-orden.md`, `docs/fase-5-division-de-trabajo.md` y `docs/fase-5-conciliacion.md` si faltan
`docs/plan-de-trabajo.md`	Decisiones 33 a 46; estado de la Fase 5 (solo después de verificarla); la fila de la Fase 4 (dice "espera PR a main") y una sección 6.2 con la Fase 5 en detalle
`docs/guia-estudio-src.md`	Archivos nuevos (si faltan filas), tabla de endpoints y fecha de actualización
PDF	Generar el de la Fase 5 y regenerar los de la guía de `src` y de esta división (ver `docs/pdf/README.md`)

No se modifica nada de `CLAUDE.md` ni del plan fuera de esta lista.

Apartado G — Pull Request

Abrí el PR como **borrador**, con la descripción de los cambios por parte y cómo se probó. **Martín**, **Contreras** y **Calvo** lo revisan y aprueban (tu aprobación no cuenta). Sin las carpetas `.postman/` ni `postman/`. El *push* a `main` dispara el despliegue automático de la VM: después del merge, `git checkout main` y `git pull`.

Criterio de terminado

- Carpetas 05 y 06 de Postman pasan con newman y las carpetas 00 a 04 siguen pasando (resultados reales registrados).
- Las pruebas de concurrencia están hechas y registradas.
- Auditoría con la matriz completa contra el PDF oficial y los defectos (si hay) registrados.
- Informe, guía, actualización del apartado F y PDF hechos y revisados contra el código.
- PR abierto como borrador, sin `.postman/` ni `postman/`.

Prompt para Claude Code (Parte 2)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 5 (punto 5: pesaje final y conciliación), rama fase-5-conciliacion. El trabajo es SECUENCIAL: primero Calvo y después Alasino (esta parte, que cierra la fase y el segundo parcial). Martín González autorizó el planeamiento de la fase el 02/11/2026 y confirmó las decisiones D1 a D14 del documento de división, los commits locales atómicos y la actualización de la documentación al cerrar la fase.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md, docs/fase-4-cierre-orden.md (como molde del informe y la guía) y docs/fase-5-division-de-trabajo.md completo (en especial las secciones 3, 6 y "Parte 2" y el bloque "Estado de la Parte 1"), y docs/pdf/consigna-trabajo-integrador-2026.pdf completo (las 9 páginas: es la fuente que manda). Revisá la colección docs/postman/IW3-Gas-Terminal.postman_collection.json (en especial las carpetas 03 y 04) para imitar su estilo.

Soy Benjamín Alasino y hago la PARTE 2 (Postman, concurrencia, auditoría final, documentación y PR). Antes de empezar hacé git fetch, git checkout fase-5-conciliacion y git pull --rebase, y comprobá que Calvo (Parte 1) ya publicó: POST /api/v1/integration/tms/orders/{n}/final-weight y GET /api/v1/orders/{n}/reconciliation funcionando, los arreglos de la tara infinita (H-26) y de activate con bloqueo (H-25), los dos campos nuevos de Order y los tests nuevos. Si falta algo, avisame y esperá.

Tu trabajo, en este orden:
1. Postman, carpeta "05 - Fase 5: pesaje final y conciliación (TMS)" con VARIABLES PROPIAS (orderNumberFinal, activationPasswordFinal y las que necesites; NO uses variables de las carpetas 00 a 04). Cubrí: preparación (crear orden, tara 14500, activar, lecturas masa 1000/densidad 0.5/temperatura 20/caudal 3000 y masa 2000/0.5/22/3600, cerrar); pesaje final inválido ({} , peso 0, negativo, 1e999 y 14000 menor que la tara: todos 400 con fragmentos, y la orden sigue en estado 3 con finalWeight nulo); orden inexistente 404 y sin token 401; pesaje final 16490.0 (200 con la conciliación: loadedProduct 2000.0, scaleNet 1990.0, difference -10.0, averageTemperature 21.0, averageDensity 0.5, averageFlowRate 3300.0, comparando con closeTo), la orden en estado 4 con finalWeight y finalWeighingDateTime, el historial con 4 registros (el último 3->4, usuario admin, "Pesaje final registrado") y el GET de la conciliación con los mismos valores; estado 4 final (segundo pesaje, lectura, activación, cierre y tara: 409); conciliación fuera del estado 4 (órdenes en estado 1, 2 y 3: 409; inexistente 404; sin token 401); pesaje final en órdenes en estado 1 y 2 (409); el límite de una orden sin lecturas con peso igual a la tara (200, loadedProduct 0.0, scaleNet 0.0, difference 0.0 y promedios null); y la regresión H-26 (tara con 1e999 en una orden en estado 1: 400 y la orden sigue en estado 1 con tara nula). Aserciones sobre mensajes con fragmentos (include) según la sección 3.6. No modifiques las carpetas 00 a 04.
2. Postman, carpeta "06 - Circuito completo (1 a 4)": un recorrido ENCADENADO de una orden con Tests de Postman, variables propias (orderNumberCircuit, activationPasswordCircuit, ...) y su propio login: Punto 1 (crear 201 y reenviar 302), Punto 2 (tara y estado 2), Punto 3 (activación con preset, cuatro lecturas válidas y una descartada, cabecera con los últimos valores), Punto 4 (cierre, estado 3 y una lectura tardía 409), Punto 5 (pesaje final; la conciliación se calcula en el test a partir de los datos enviados y se compara con la respuesta con closeTo; el GET coincide con el POST), historial con exactamente 4 registros (null->1, 1->2, 2->3, 3->4, con el usuario) y la orden presente en GET /orders y en GET /orders/{n}/details. Cada pedido guarda en variables lo que usa el siguiente. Commit del JSON aparte, solo ese archivo, y verificá con git diff que solo cambie lo nuevo.
3. Corré todo: npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json y ./mvnw -B test (con la base arriba, preferentemente una base descartable iw3_scratch con la app en el puerto 8089). Tienen que seguir pasando las carpetas 00 a 04 (86 pedidos y 204 aserciones). Registrá comandos y resultados REALES.
4. Concurrencia con un script descartable FUERA del repo y una base descartable: (a) dos pesajes finales simultáneos sobre una orden en estado 3 (un 200 y un 409, una sola fila 3->4, orden en estado 4); (b) una activación y un cierre simultáneos sobre una orden en estado 2 sin activar, repetido varias veces (H-25: nunca queda en estado 2 tras un cierre 200); (c) dos activaciones simultáneas de la misma orden (ambas 200 y activationDateTime guardado una sola vez); (d) regresión de la Fase 4: unas 50 lecturas y un cierre en paralelo (sin 500, una sola fila 2->3, cabecera = máximo guardado). Mostrame qué hiciste y qué obtuviste.
5. AUDITORÍA FINAL: auditá si el segundo parcial quedó completo. Armá una matriz de trazabilidad (requisito -> archivo y método -> prueba -> resultado -> clasificación: implementado y verificado / implementado y no verificado / parcial / con defectos / pendiente). Revisá: los Puntos 1 a 5 contra el PDF original (nombres de campos, reglas, estados y la tabla de la conciliación); los requisitos obligatorios del segundo parcial (APIs sin interfaz gráfica; Swagger/OpenAPI completo: todos los endpoints de /v3/api-docs con @Operation y @ApiResponse; Postman que pruebe TODAS las APIs: cruzá los endpoints de /v3/api-docs contra los pedidos de la colección y listá los que no estén cubiertos; persistencia; estados secuenciales; registro de cada cambio de estado; descarte de lecturas); los opcionales (circuito con Tests, promedios, frecuencia de almacenamiento: no hecha); los 21 criterios de la sección 6 con evidencia; que el código haga exactamente lo decidido en D1 a D14 (el 400 por tara después del 409, peso igual a la tara válido, conciliación no guardada, activate con bloqueo); el orden de comprobaciones y que ninguna esté dentro del try; que no haya lógica duplicada, dependencias nuevas, lista de detalles en Order ni cambios fuera de lo permitido en las Fases 1 a 4; que la contraseña no aparezca en mensajes, logs ni consultas; las regresiones; y que el estado 4 sea final (lecturas, activación, cierre, tara y pesaje final dan 409). Si encontrás un defecto, no lo arregles de entrada: registralo con el pedido exacto, la respuesta, el archivo y el método, y decime a quién corresponde; si es un cambio mínimo y el dueño lo autoriza, hacelo en un commit fix aparte explicando el motivo. No des la fase por completa si queda un criterio esencial
```

```
sin cumplir o sin verificar: decí que está parcialmente completa y qué falta.
6. Documentación: creá docs/fase-5-conciliacion.md con el formato de docs/fase-4-cierre-orden.md (informe técnico con decisiones D1 a D14 y la alternativa descartada, inventario de archivos por parte, recorridos del POST y del GET, funciones y métodos, pruebas con resultados reales, cambios de comportamiento respecto de las fases anteriores, hallazgos y limitaciones -H-26 resuelto y los nuevos desde H-27- y la matriz de auditoría; y guía de estudio A-J con 10 preguntas o más y una sección de qué debe poder explicar cada integrante), integrando el bloque de la Parte 1. La atribución es documental: distinguí responsabilidad asignada de cambios observados. Controlá que todo lo escrito coincida con el código real; si no coincide, corregí el documento y no el código.
7. Actualizá la documentación del proyecto (autorizado), en commits aparte y con cambios mínimos, SOLO lo siguiente: CLAUDE.md secciones 2 (estado real: la Fase 4 entró por el PR #12), 5 (Fase 4 integrada, bloque de la Fase 5, cifras de la carpeta 03: 45 pedidos y 117 aserciones), 6, 7 (clases, endpoints, ejemplo de la conciliación y cifras de pruebas medidas al cerrar), 8.1, 8.3 (el ruleset de main existe pero está deshabilitado), 9 (decisiones 33 a 46), 10 (H-26 y H-25 resueltos, H-16 resuelto como efecto colateral, H-12 sigue aceptado, precisiones de H-5 y los hallazgos nuevos) y 14; docs/plan-de-trabajo.md (decisiones 33 a 46, estado de la Fase 5 solo después de verificarla, la fila de la Fase 4 que dice espera PR, y una sección 6.2 de la Fase 5); docs/guia-estudio-src.md (filas, endpoints y fecha). No modifiques nada más de CLAUDE.md ni del plan. Generá docs/pdf/fase-5-conciliacion.pdf y regenerá docs/pdf/guia-estudio-src.pdf y docs/pdf/fase-5-division-de-trabajo.pdf (ver docs/pdf/README.md); agregá la fila del PDF nuevo al README.
8. Abrí el PR como borrador solo cuando yo te lo pida, sin las carpetas .postman/ ni postman/.

NO toques el código de src/main ni src/test salvo un fix mínimo y autorizado de la auditoría. No implementes la frecuencia de almacenamiento, roles, alarmas ni frontend (examen final). Si creés que hace falta un cambio ajeno, avisame en vez de hacerlo.

Reglas: documentación en español, identificadores en inglés. Commits atómicos de 100 líneas o menos (excepción: documentos completos nuevos y el JSON de Postman, un commit aparte cada uno), en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code" (están autorizados los commits locales). No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. Reportá solo lo que ejecutaste; si algo falla, decilo. Antes de editar, mostrame tu plan y esperá mi OK.

Al terminar: agregá al final de docs/fase-5-division-de-trabajo.md el bloque "Estado de la Parte 2" (qué quedó hecho, commits, resultados reales de las pruebas y de la concurrencia, el resultado de la auditoría y los defectos o pendientes) y las filas de tus archivos en docs/guia-estudio-src.md marcadas F5 (commits aparte). No pushees hasta que yo lo pida.
```

Cierre de la fase

Checklist de integración

✓	Verificación	Quién
☐	`./mvnw -B -DskipTests compile` sin errores en la rama	Todos
☐	`./mvnw -B test` pasa (con la base arriba): los 19 previos más los nuevos	Calvo y Alasino
☐	La app arranca contra una base que ya tenía órdenes y agrega las dos columnas	Calvo
☐	Arreglos H-26 (tara infinita → 400) y H-25 (`activate` con bloqueo) con sus tests	Calvo
☐	Los 21 criterios de la sección 6, con evidencia	Alasino
☐	newman: carpetas 05 y 06 en verde y carpetas 00 a 04 intactas (86 pedidos, 204 aserciones)	Alasino
☐	Pruebas de concurrencia registradas (pesaje final doble, activación vs. cierre, doble activación)	Alasino
☐	Auditoría final contra el PDF oficial, con matriz completa y defectos registrados	Alasino
☐	Actualización de `CLAUDE.md` , plan y guía de `src` (apartado F) y PDF regenerados	Alasino
☐	Revisión cruzada (Alasino audita a Calvo; Calvo revisa a Alasino)	Los dos
☐	PR abierto como borrador, sin `.postman/` ni `postman/`	Alasino
☐	Aprobaciones de Martín, Contreras y Calvo antes de mergear	Equipo
☐	Después del merge: el workflow de deploy redespliega la VM; comprobar que terminó en verde	Equipo

Problemas frecuentes

Síntoma	Causa probable	Qué hacer
El push es rechazado	Alguien pusheó antes	`git pull --rebase` y volver a pushear
El contexto de Spring no arranca (dependencia circular)	`OrderBusiness` inyecta `IOrderDetailBusiness`	Inyectar `OrderDetailRepository`
500 en vez de 400, 404 o 409	La comprobación quedó dentro del `try/catch`	Moverla fuera del `try`
500 con `NullPointerException` en el pesaje final de una orden en estado 1	Se compara con la tara antes de comprobar el estado	Comprobar el estado (409) antes de comparar con la tara
Los promedios dan error con cero lecturas	La proyección no coincide con los alias, o se espera un valor no nulo	Alias iguales a los getters; tratar `null` como "sin promedio"
Una aserción de Postman falla por `3900.1000000000004`	Comparación exacta de `double`	Usar `closeTo` o valores exactos
Falla una aserción de la carpeta 02	Se cambió el texto `weight debe ser mayor que 0 (kg)`	Dejar ese texto y usar otro solo para el infinito
Falla una aserción de una carpeta anterior	Se usó una variable de otra carpeta	Usar variables propias
Un test de Mockito falla con `UnnecessaryStubbingException`	Un *stub* que el test no usa	Quitarlo o usar `lenient()` solo si hace falta
401 en todo	Token vencido (dura 1 hora) o falta el usuario de prueba	Volver a loguearse / correr el seed
El contenedor de la base no arranca	Docker apagado	Iniciar Docker; el contenedor del TP es `iw3-gas-mysql`

Después de la Fase 5

Con la Fase 5 terminan los **Puntos 1 a 5** y los requisitos obligatorios del **segundo parcial** (entrega el **10/11/2026**). Quedan para el **examen final**: roles por endpoint, alarma de temperatura por e-mail con umbral y destinatarios configurables y sin reenvío hasta ser aceptada, frontend (login, monitoreo en tiempo real, ETA, tiempo transcurrido, alarmas y conciliaciones), endurecimiento de seguridad (H-5) y la frecuencia de almacenamiento configurable (opcional).

Anexo A — Hallazgos del diagnóstico previo

Registrados al preparar esta fase (02/11/2026), cada uno verificado contra el código, la consigna o el repositorio:

#	Hallazgo	Qué se hace
1	**Decisión 30 y producto cargado nulo.** Una orden cerrada sin activar y sin lecturas llega al estado 3 con `lastAccumulatedMass` nulo (probado el 02/11/2026: queda en estado 3 con 3 filas de historial). Está anotado en la guía de inicio de la Fase 4	**D6**: producto cargado 0.0 y promedios `null`, sin bloquear el pesaje final. D2 permite peso igual a la tara
2	**H-25.** `OrderBusiness.activate` busca sin bloqueo y guarda con `saveAndFlush` (escribe todas las columnas): una primera activación simultánea con el cierre podría reescribir el estado 2. Confirmado por código; no reproducido	**D14**: bloqueo en `activate` (Parte 1). Resuelve también H-16. Se verifica en la Parte 2

#	Hallazgo	Qué se hace
3	**H-26 (nuevo).** `POST .../tare` con `{"weight": 1e999}` responde 500 con el formato por defecto de Spring (con el `trace`); la orden queda en estado 1 con tara nula. Es el mismo defecto que F-02 en las lecturas. **Verificado** el 02/11/2026 contra una base descartable	**D13**: `requireValidWeight` en la tara y en el pesaje final (Parte 1)
4	**La Fase 4 ya está en `main`** (PR #12, mergeado el 31/10/2026 con una sola aprobación). `CLAUDE.md` (secciones 2, 5 y 8.1) y la fila 4 del plan todavía dicen "espera PR"	Lo corrige Alasino al cerrar (apartado F de la Parte 2)
5	**Protección de `main`.** Existe un *ruleset* ("iw3 - Estandar de Repositorios") pero su enforcement está **deshabilitado**: GitHub no exige las 3 aprobaciones de la decisión 14. El equipo lo había dado por no necesario	No se cambia. El acuerdo de las 3 aprobaciones se cumple a mano; se documenta en `CLAUDE.md` 8.3
6	**H-5 y el despliegue automático.** Cada push a `main` despliega en la VM (`.github/workflows/deploy.yml`). Siguen la clave JWT fija (`AuthConstants.SECRET`) y `/demo/**` público. `dev.info.enabled` es `true` en el perfil por defecto y en `mysqldev` y `false` en `mysqlprod` (`CLAUDE.md` lo da como `true` en general); `mysqlprod` trae una contraseña por defecto escrita en el `.properties`; el chequeo del deploy llama a `/api/v1/products`, una ruta retirada en la Fase 1 (pasa porque responde 401)	Se precisa H-5 en `CLAUDE.md` (apartado F). Se cierra en la etapa de seguridad del final
7	**La consigna en PDF ya está en el repo** (`docs/pdf/consigna-trabajo-integrador-2026.pdf`). Contrastada: Punto 5, estados, notas, parcial y final coinciden con `CLAUDE.md` sección 4. Un matiz: el opcional de Postman dice "hasta el cierre"; la carpeta 06 llega al estado 4	Alasino audita contra el PDF (apartado D de la Parte 2)
8	**"Decisión 11" tiene dos significados** (`CLAUDE.md` sección 9: `/error` y el login; división de la Fase 3: el histórico paginado, que es la decisión 28)	Este documento usa solo los números globales (33 a 46)
9	**El historial de `main` está duplicado.** El merge `ec6aa50` (02/11/2026) juntó dos copias de los mismos commits con hashes distintos (257 asuntos repetidos). El contenido es idéntico: `git diff` vacío	No se toca. Si molesta, decidir entre todos y sin reescribir `main`
10	**Cifras de `CLAUDE.md` desactualizadas:** la carpeta 03 de Postman figura con 44 pedidos y 114 aserciones y tiene **45 y 117**	Lo corrige Alasino (apartado F)
11	**H-12** (taras simultáneas sobre la misma orden) sigue aceptado: `registerTare` busca sin bloqueo. Solo se toca el chequeo del peso (D13)	Se registra; fuera de alcance
12	`.postman/` y `postman/` aparecen como carpetas sin seguimiento y no están en `.gitignore`	No se commitean

Anexo B — Registro de traspasos

Cada responsable completa su fila al terminar y **agrega debajo su bloque "Estado de la Parte N"**.

Parte	Responsable	Estado	Último commit	Fecha
1	Tomás Calvo	Completa y verificada (local, sin push)	`7408b07` (último commit de código)	02/11/2026
2	Benjamín Alasino	Completa y verificada (local, sin push; PR pendiente)	`bcda1b3` (antes del bloque de estado)	02/11/2026

Modelo del bloque "Estado de la Parte N"

```
## Estado de la Parte N (dd/mm/aaaa)

| Paso | Commit | Estado |
|---|---|---|
| ... | `abc1234` `tipo(ámbito): descripción` | Pusheado / Local |

**Qué quedó implementado** (archivos y qué hace cada uno).

**Verificado** (comandos y resultados reales; marcá lo que NO pudiste ejecutar).

**Para tener en cuenta** (lo que necesita saber el siguiente: limitaciones, decisiones, casos probados,
mediciones).
```

Estado de la Parte 1 (02/11/2026)

Paso	Commit	Estado
Campos del pesaje final en `Order`	`4131c00` `feat(model): agrega a la orden los campos del pesaje final`	Local
Consulta de promedios (`ReadingAverages` , `averagesByOrder`)	`ff05d96` `feat(model): agrega la consulta de promedios de las lecturas de una orden`	Local
`Reconciliation`	`6742511` `feat(business): agrega Reconciliation, la respuesta de la conciliación`	Local
Arreglo H-26 (D13)	`8ea3244` `fix(business): rechaza el peso infinito en la tara con 400 (H-26)`	Local
Test de la tara	`6a1a558` `test(business): prueba la validación del peso de la tara (H-26)`	Local
Arreglo H-25 (D14)	`59a79b9` `fix(business): busca la orden con bloqueo en la activación (H-25)`	Local
Tests de la activación (2 commits)	`d4274f7` , `a5e76aa`	Local
Interfaz + `registerFinalWeight` + `buildReconciliation`	`eb5a203` `feat(business): agrega el pesaje final y la función compartida de la conciliación`	Local
`reconciliation`	`6a876a8` `feat(business): agrega la consulta de la conciliación para órdenes en estado 4`	Local
Tests del pesaje final y la conciliación (4 commits)	`f57dc6c` , `7e1fab3` , `1bd80d0` , `736f027`	Local
`FinalWeightRequest`	`867f4a9` `feat(integration): agrega el pedido del pesaje final de la balanza`	Local
Endpoint del pesaje final	`6946d85` `feat(integration): agrega el endpoint del pesaje final de la balanza`	Local
Endpoint de la conciliación	`7408b07` `feat(controllers): agrega el endpoint de la conciliación`	Local
Filas de la guía de `src` (F5)	`f16930d` `docs(guia): agrega al mapa de código los archivos de la Parte 1 de la Fase 5`	Local

No hay push todavía. Todos los commits tienen 100 líneas o menos (el test de la activación se partió en dos después de medirlo: tenía 124). La interfaz de `IOrderBusiness` se partió por método y va junto con su implementación (un commit con solo la interfaz no compila). El test del pesaje final se partió en 4 commits para no pasar las 100 líneas. El PDF de este documento no se regeneró.

Qué quedó implementado (rutas bajo `src/main/java/ar/edu/iua/iw3/`):

- `model/Order.java`: `finalWeight` (`Double`) y `finalWeighingDateTime` (`LocalDateTime`), opcionales y `READ_ONLY`. No tiene lista de detalles.
- `model/persistence/ReadingAverages.java` y `OrderDetailRepository.averagesByOrder`: `AVG` simple de temperatura, densidad y caudal de las lecturas de una orden.
- `business/Reconciliation.java`: 9 campos con `@Schema`; es el cuerpo de la respuesta del POST y del GET.
- `business/OrderBusiness.java`:
  - `requireValidWeight` (H-26): `!(weight > 0)` → 400 con el texto de siempre; infinito → 400 "weight debe ser un número finito". `registerTare` lo usa; el resto de `registerTare` no cambió.
  - `activate` (H-25): usa `loadForUpdate` después de comprobar los nulos; el orden 400 → 404 → 409 → 403, el 403 sin contraseñas y la idempotencia no cambiaron.
  - `registerFinalWeight`: `loadForUpdate` (404) → `requireValidWeight` (400) → estado 3 (409) → tara nula (409) → peso < tara (400), todo fuera del `try`. Dentro: guarda el peso y la hora del servidor, estado 4, `saveAndFlush` y una fila 3 → 4.
  - `reconciliation`: `load` (404) → estado 4 (409) → `buildReconciliation`. Sin bloqueo.
  - `buildReconciliation` (privado): una sola función para el POST y el GET. Sin redondeo; no guarda nada. `OrderDetailRepository` va inyectado directo (no `IOrderDetailBusiness`, por la dependencia circular).
- `integration/tms/model/FinalWeightRequest.java` (`double weight`) y `POST /api/v1/integration/tms/orders/{orderNumber}/final-weight` en `OrderTmsRestController`; `GET /api/v1/orders/{orderNumber}/reconciliation` en `OrderRestController`. Los métodos existentes no se tocaron.
- Tests (Mockito): `OrderBusinessTareTest` (4), `OrderBusinessActivateTest` (6), `OrderBusinessFinalWeightTest` (11).

Verificado (02/11/2026, MySQL real en el contenedor `iw3-gas-mysql`):

Verificación	Resultado
`./mvnw -B -q -DskipTests compile` tras cada commit de código	OK
`./mvnw -B test`	**40 tests, 0 fallas** (los 19 previos + 4 + 6 + 11), `BUILD SUCCESS`
Los tests detectan los defectos	Con el código anterior al arreglo, `OrderBusinessTareTest` falla 1 de 4 (`BusinessException` en lugar de `BadRequestException`) y `OrderBusinessActivateTest` falla 5 de 6
Esquema sobre una base con órdenes	Base descartable `iw3_scratch` clonada de `iw3_db` (1 orden y el usuario `admin`), sin las columnas nuevas. Al arrancar (`Started Iw3Application in 9.412 seconds`, 0 `ERROR`) Hibernate agregó `final_weight` (`double`) y `final_weighing_date_time` (`datetime`), ambas nulables. La orden previa quedó intacta
Script de verificación contra la app en el puerto 8089	**43 PASS, 0 FAIL** (casos abajo)
Cruce con la base	Una sola fila 3 → 4 por orden finalizada (2 en toda la base). La orden con tara `1e999` sigue en estado 1 con tara nula y sin fila nueva. La orden cerrada sin lecturas tiene 0 filas en `order_details`
`npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json --env-var "baseUrl=http://localhost:8089"`	86 pedidos, **204 aserciones, 0 fallas**
`/v3/api-docs`	`final-weight` con 200/400/404/409 y `reconciliation` con 200/404/409; esquemas `Reconciliation` (9 campos) y `Order` con los dos campos nuevos

Casos comprobados con pedidos reales (orden con tara 14500 y lecturas 1000/0.5/20/3000 y 2000/0.5/22/3600, ya cerrada):

- `{}`, peso 0 y negativo → 400 "weight debe ser mayor que 0 (kg)". Peso `1e999` → 400 "weight debe ser un número finito" con `StandartResponse`. Peso 14000 → 400 "weight (pesaje final) no puede ser menor que la tara (14500.0 kg)". La orden sigue en estado 3 con `finalWeight` nulo.
- Orden inexistente → 404. Estado 1 y 2 → 409 "no admite pesaje final: debe estar en estado 3 (estado N)". Sin token → 401.
- Peso 16490.0 → 200 con `loadedProduct` 2000, `scaleNet` 1990, `difference` -10, `averageTemperature` 21, `averageDensity` 0.5, `averageFlowRate` 3300. `GET /orders/{n}`: estado 4, `finalWeight` 16490 y `finalWeighingDateTime` del servidor. `/status-log`: 4 registros; el último 3 → 4, usuario `admin`, "Pesaje final registrado".
- Segundo pesaje → 409. Con la orden en estado 4: lectura, activación, cierre y tara → 409.
- `GET /reconciliation`: estado 4 → 200 con los mismos valores del POST; estados 1, 2 y 3 → 409 "solo está disponible para órdenes en estado 4"; inexistente → 404; sin token → 401.
- Orden cerrada **sin lecturas** con peso 14500 (igual a la tara) → 200: `loadedProduct` 0.0, `scaleNet` 0.0, `difference` 0.0 y los tres promedios `null`; el GET devuelve lo mismo.
- H-26: `POST .../tare` con `1e999` → 400; la orden queda en estado 1 con tara nula. Tara 0 → sigue el mensaje original.
- Activación: `{}` y sin `password` → 400, inexistente → 404, estado 1 → 409, contraseña incorrecta → 403 sin contraseñas en el cuerpo, repetirla → 200 con el mismo preset y la misma `activationDateTime`.

Promedios con cero lecturas: el `AVG` no lanza excepción. La base devuelve una fila con `NULL` y la app responde 200 con los tres promedios `null`. No distinguí si el repositorio devuelve una proyección con valores nulos o directamente `null`; `buildReconciliation` trata los dos casos igual.

No ejecutado o no verificado:

- **Concurrencia** (criterios 12 y 14): dos pesajes finales simultáneos y la activación simultánea con el cierre **no se probaron**; es trabajo de la Parte 2. Hoy solo hay evidencia de que `activate` y `registerFinalWeight` usan la consulta con bloqueo (tests de Mockito).
- Swagger UI en el navegador (solo el JSON de `/v3/api-docs`) y el despliegue en la VM.
- El PDF de este documento y del resto de la documentación (lo hace Alasino al cerrar).

Casos que Alasino debe pasar a Postman (carpeta 05; la 06 es el circuito encadenado):

1. Preparación: orden en estado 3 (crear, tara 14500, activar, lecturas 1000/0.5/20/3000 y 2000/0.5/22/3600, cerrar), una orden en estado 1 y una en estado 2.
2. Pesaje final inválido (la orden sigue en estado 3): `{}`, peso 0, negativo, `1e999` (cuerpo crudo `{"weight": 1e999}`) y 14000. Después un `GET` que compruebe estado 3 y `finalWeight` nulo.
3. 404 (inexistente), 409 (estado 1 y 2), 401 (sin token).
4. Peso 16490.0 → 200 con los valores del ejemplo; `GET /orders/{n}` con estado 4, `finalWeight` y `finalWeighingDateTime`; `status-log` con 4 registros y el último 3 → 4; `GET /reconciliation` con los mismos valores; segundo pesaje → 409.
5. `GET /reconciliation` en estados 1, 2 y 3 → 409, inexistente → 404, sin token → 401.
6. Estado 4: lectura, activación, cierre y tara → 409.
7. Orden cerrada sin lecturas con peso igual a la tara → 200, `loadedProduct` 0.0 y promedios `null`.
8. H-26: tara `1e999` sobre una orden en estado 1 → 400 y la orden sigue en estado 1 con `tare` nula.
9. Activación: sin campos, sin `password`, inexistente, estado 1, contraseña incorrecta, repetida (misma `activationDateTime`).

Para tener en cuenta (Alasino):

- **Mensajes exactos** (para los `include`): los de la sección 3.6; los verifiqué tal cual. En la carpeta 02 el mensaje de "≤ 0" no cambió.

- **El orden de D2 tiene un 400 después del 409.** Un peso inválido (0, negativo, infinito) sobre una orden en estado 1 o 2 da **400**, no 409, porque el peso se valida antes que el estado. Está en el código y en los tests unitarios; **no lo probé con un pedido real**.
- **Valores exactos.** Los ejemplos de la sección 3.5 son exactos en `double`; para otros valores usar `closeTo`. Los números salen sin redondeo.
- `finalWeighingDateTime` es la hora del servidor: no se puede comparar con un valor fijo; verificar que sea un texto.
- **Base `iw3_db`:** `contextLoads` corre contra ella y ya le agregó las dos columnas nuevas (nulables, inofensivas). Para repetir la prueba de esquema hace falta una base sin ellas.
- **Docker en Windows:** el contenedor `iw3-gas-mysql` arrancó otra vez sin publicar el puerto 3306 al host (`docker ps` mostraba solo `3306/tcp`). Se arregló con `docker compose up -d --force-recreate`, que conserva los datos. Además estaba `Exited (128)` y había otro contenedor (`mysqliw3`, de la cátedra).
- **newman contra otro puerto:** `--env-var "baseUrl=http://localhost:8089"` sobrescribe la variable de la colección.
- **Hallazgos sin tocar:** H-12 (taras simultáneas) sigue aceptado; H-21 y H-23 (números como texto y rutas no numéricas) no cambian: un `weight` `"abc"` sigue saliendo con el formato por defecto de Spring.
- **Atribución:** todos los commits de esta parte son de `tomas-calvo01`. De las fases anteriores solo se tocaron `registerTare` (D13) y `activate` (D14), con la autorización de Martín González.

Estado de la Parte 2 (02/11/2026)

Paso	Commit	Estado
Carpetas Postman 05 y 06 (solo el JSON; `git diff --numstat`: 3142 agregadas, 0 borradas)	`0622a20` `test(postman): agrega las carpetas 05 (pesaje final y conciliación) y 06 (circuito completo)`	Local
Informe, auditoría final y guía de estudio	`1c281f0` `docs(fase-5): agrega el informe, la auditoría final y la guía de estudio de la Fase 5`, `1b2c049` (aclaración de la sección K)	Local
`CLAUDE.md` secciones 2 y 5	`0a643b7`	Local
`CLAUDE.md` secciones 6 y 7	`0cf47b7`	Local
`CLAUDE.md` secciones 8.1 y 8.3	`cfc5fa3`	Local
`CLAUDE.md` sección 9 (decisiones 33 a 46)	`8f01997`	Local
`CLAUDE.md` secciones 10 y 14	`ecdd957`	Local
Plan: decisiones 33 a 46 · filas de las Fases 4 y 5 y sección 6.2	`b3082fa`, `6a0669f`	Local
Guía de `src`: fecha, fases, endpoints y códigos · filas F5 de la Parte 2	`014d87c`, `194f79f`	Local
PDF de la Fase 5 y fila en el índice · PDF de la guía de `src`	`ac69aa4`, `bcda1b3`	Local
Este bloque y el PDF de este documento	Commits `docs(fase-5)` y `docs(pdf)` siguientes	Local

Todos los commits tienen 100 líneas o menos, salvo las dos excepciones acordadas (el JSON de Postman y el informe nuevo completo, cada uno en su commit). Autor: `Benjamin Alasino`; ningún mensaje lleva `Co-Authored-By` ni "Generated with Claude Code". **No hay push**: se hace cuando Benjamín lo pida, después de `git pull --rebase`.

Punto de partida verificado. `git fetch`, `git checkout fase-5-conciliacion` y `git pull --rebase`: la rama local coincidía con `origin/fase-5-conciliacion` (`9347a95`). La Parte 1 **sí estaba publicada** en `origin`, aunque su bloque diga "Local / sin push" (el bloque quedó escrito antes del push; no se modificó porque es de Calvo).

Qué quedó hecho. Solo documentación y la colección Postman: la Parte 2 **no modificó** `src/main` ni `src/test`. El generador de las carpetas 05 y 06 y el script de concurrencia fueron descartables y quedaron fuera del repo.

- **Carpeta 05** (44 pedidos, 98 aserciones; variables `orderNumberFinal`, `activationPasswordFinal`, `reconciliationFinal`, `orderNumberFinalPending`, `orderNumberFinalTared`, `orderNumberFinalBare`, `reconciliationFinalBare`, `orderNumberFinalTare`): preparación del ejemplo, 5 pesajes inválidos (400) con la orden intacta y sin fila nueva, 404, 401, pesaje final 16490.0 con `closeTo`, estado 4, historial de 4 filas, GET = POST, estado 4 final (5 × 409), conciliación en estados 1, 2 y 3 (409), pesaje final en estados 1 y 2 (409), **peso 0 sobre una orden en estado 1 (400: el peso se valida antes que el estado, D2)**, orden sin lecturas con peso igual a la tara y regresión H-26.
- **Carpeta 06** (21 pedidos, 56 aserciones; login propio con `tokenCircuit` y auth Bearer a nivel de carpeta): Puntos 1 a 5 encadenados, 302 al reenviar, 4 lecturas válidas no enteras y una descartada, cabecera, lectura tardía 409, conciliación **recalculada en el test** con `closeTo`, GET = POST, historial exacto `null→1, 1→2, 2→3, 3→4` con el usuario, presencia en `GET /orders` (recorre páginas con `pm.sendRequest` si hace falta) y en `GET /orders/{n}/details`.

Verificado (02/11/2026, base descartable `iw3_scratch` con la app en el puerto 8089; al terminar se bajó la app y se borró la base):

Verificación	Resultado
`npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json --env-var "baseUrl=http://localhost:8089"`	**151 pedidos, 358 aserciones, 0 fallas**. Carpetas 00 a 04: **86 y 204** (sin cambios); 05: 44 y 98; 06: 21 y 56
Carpetas por separado	00 + 05: 46 y 102, 0 fallas · 06 sola (sin la 00): 21 y 56, 0 fallas
`./mvnw -B test`	**40 tests, 0 fallas**, `BUILD SUCCESS`
(a) Dos pesajes finales simultáneos, 5 veces	Siempre un 200 y un 409; estado 4; 1 fila 3 → 4
(b) Activación y cierre simultáneos (orden en estado 2 sin activar), 20 veces	18 × (200, 200) y 2 × (409, 200); **estado final 3 en las 20**; 1 fila 2 → 3
(c) Dos activaciones simultáneas, 5 veces	Ambas 200; *general log* de MySQL: **1** `UPDATE` con `activation_date_time` por orden
(d) 50 lecturas y un cierre en paralelo, 2 corridas	25 × 200 y 25 × 409, **0 × 500**; cierre 200; 1 fila 2 → 3; cabecera 550 = máximo 550; 0 lecturas posteriores al cierre
Log de la app	0 `ERROR`, ningún *deadlock*. **0 apariciones** de las contraseñas generadas
`/v3/api-docs` contra la colección	12 de 13 endpoints con pedidos; falta `GET /demo/encodepass` (H-28)

Honestidad sobre la concurrencia: la primera corrida del script tenía dos errores propios (el número de orden no avanzaba porque se calculaba en una *subshell*, y no se extraía la contraseña de la respuesta en varias líneas). Se descartó y se repitió completa con el script corregido; los números de arriba son de esa segunda corrida. Una pausa de ~1 min 47 s en (b) fue del proceso o del equipo (`HikariPool … clock leap detected`), no de la base.

Resultado de la auditoría (detalle y matriz en `docs/fase-5-conciliacion.md`, sección 9): los **21 criterios** de la sección 6 se cumplen con evidencia. Los Puntos 1 a 5 contra el PDF y D1 a D14 contra el código coinciden. Las comprobaciones están fuera del `try`, la función de cálculo es una sola, no hay dependencias nuevas ni lista de detalles en `Order`, y los únicos cambios en las Fases 2 y 3 son los de D13 y D14. **Fase 5: completa. Requisitos obligatorios del segundo parcial para los Puntos 1 a 5: implementados y verificados.**

Defectos y pendientes (ninguno bloquea; no se cambió código):

- **H-27** (documentación): `POST /login` y `GET /demo/encodepass` sin `@Operation`; `GET /orders/{n}` y `/status-log` no declaran el 404; solo el cierre declara 401 y 500 (tampoco `final-weight` ni `reconciliation`). Arreglo propuesto: anotaciones. Lo decide el equipo; corresponde a los autores de cada endpoint (para los de la Fase 5, Calvo).

- **H-28** (Postman): `GET /demo/encodepass` (demo de la cátedra) sin pedido. Propuesta: retirarlo con H-5; si se conserva, agregar un pedido (Parte 2).
- **H-29** (Fase 2): `registerTare` guarda con `save` y no con `saveAndFlush`. Deuda menor, fuera del alcance de D13.
- Siguen: decisión 30 a confirmar, H-12 aceptado, H-21 y H-23 sin cambios.
- **PR a `main`** como borrador: cuando Benjamín lo pida, sin `.postman/` ni `postman/` (no existen en esta copia).